人工智能与深度学习

大数据与人工智能

华中科技大学 机械科学与工程学院
陈伟

2026

大数据的基本概念

大数据的概念

- 多大的数据算大数据?



traditional
computer science

data that will not fit
in main memory.

data with a *large*
number of observations
and/or features.



statistics





other fields

non-traditional sample size
(i.e. > 100 subjects); can't
analyze in stats tools (Excel).

From H.A. Schwartz's course slides

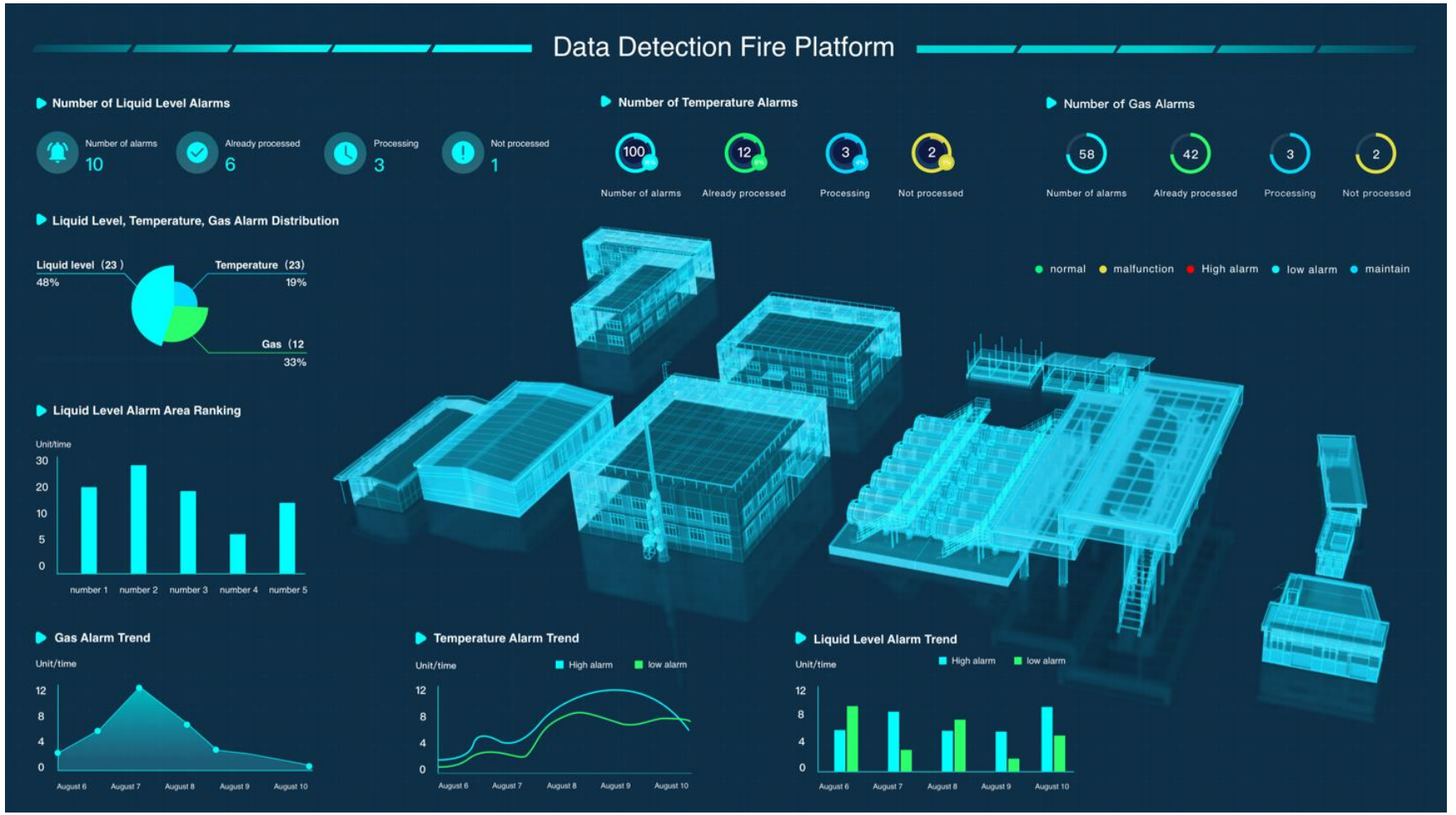
大数据的概念

- 大数据 vs 大看板（dashboard）



大数据的概念

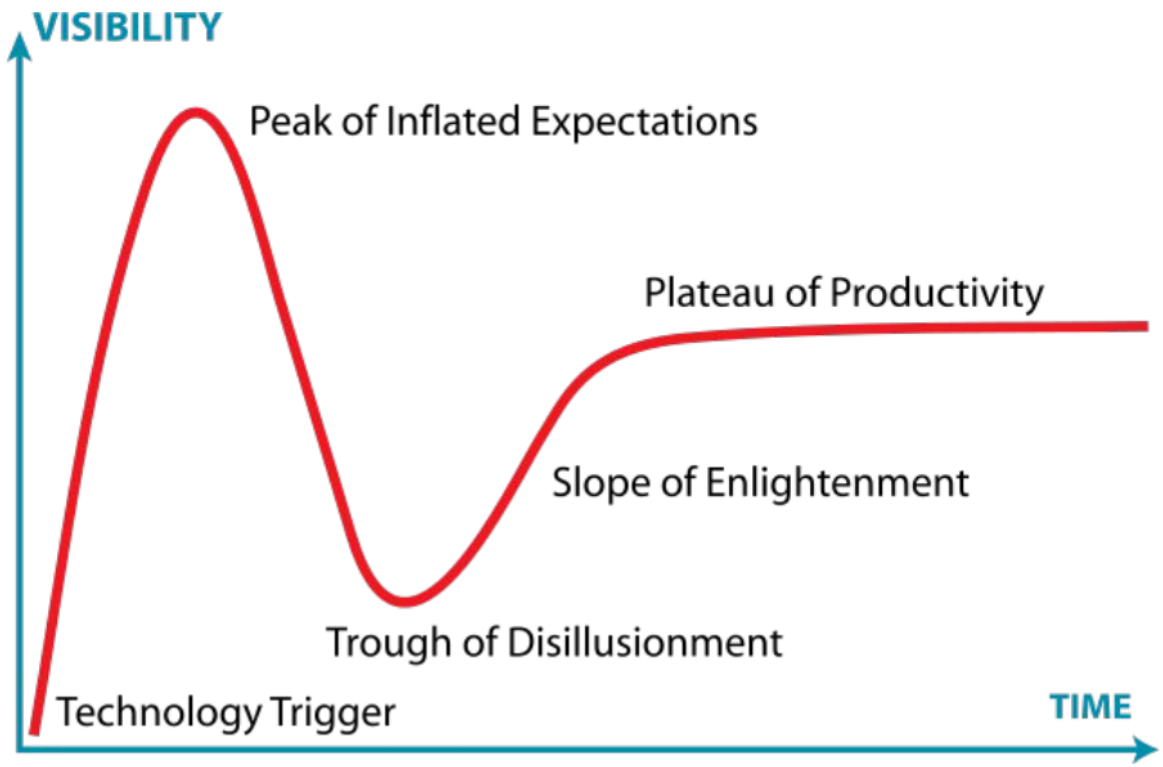
- 大数据 vs 大看板（dashboard）



大数据的概念

- **一般认为，大数据有以下这些特点**
  - Volume：数据量大
  - Variety：数据类型繁多
  - Velocity：数据速度快

大数据的发展历程



(Gartner Hype Cycle)

From H.A. Schwartz's course slides

大数据的发展历程

-



From H.A. Schwartz's course slides

大数据的发展历程

- 

(Gartner Hype Cycle)

From H.A. Schwartz's course slides

大数据与人工智能

- **大数据可以支持人工智能的发展**
  - 大数据可以用于机器学习的训练
  - 大数据可以用于发掘人工智能应用的方向

- **人工智能可以推进大数据应用，发挥大数据价值**
  - 数据 + 智能 = 价值

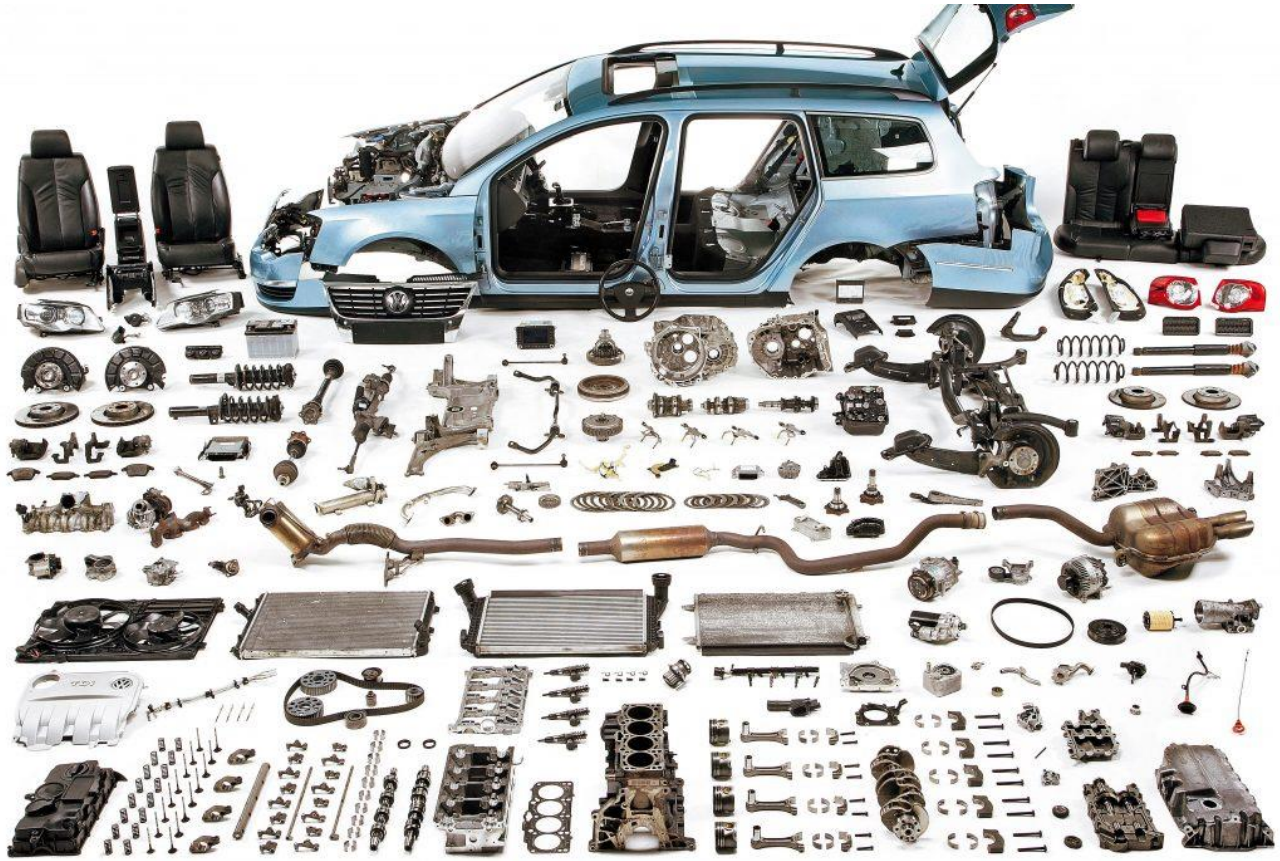
大数据的误区

- 所有数据都是有价值的；数据越多，价值越高 ✗
- 大数据就是做看板 ✗
- 大数据可以解决许多专业/业务解决不了的问题 ✗
- 大数据可以独立于已有的业务，单独发挥其价值 ✗

大数据示例

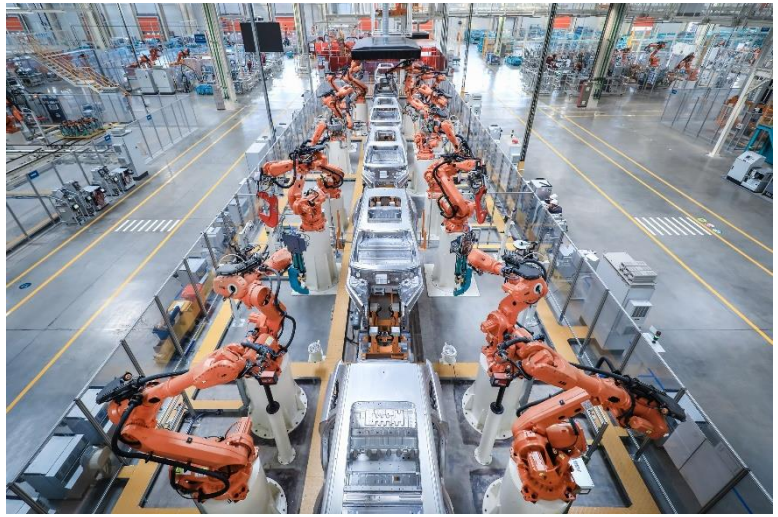
智能制造大数据

- 以汽车制造为例：一辆汽车超过1万个零件



智能制造大数据

- **汽车整车制造的企业，一般只从事汽车总装及车身制造**
  - 传统汽车制造四大生产工艺：冲压—焊装—涂装—总装
  - 电动汽车将电池总成制造纳入第五大工艺



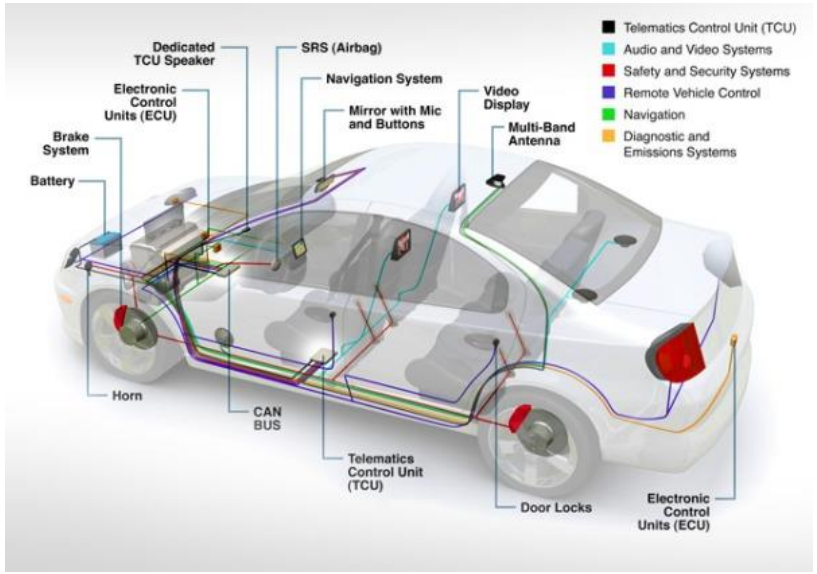


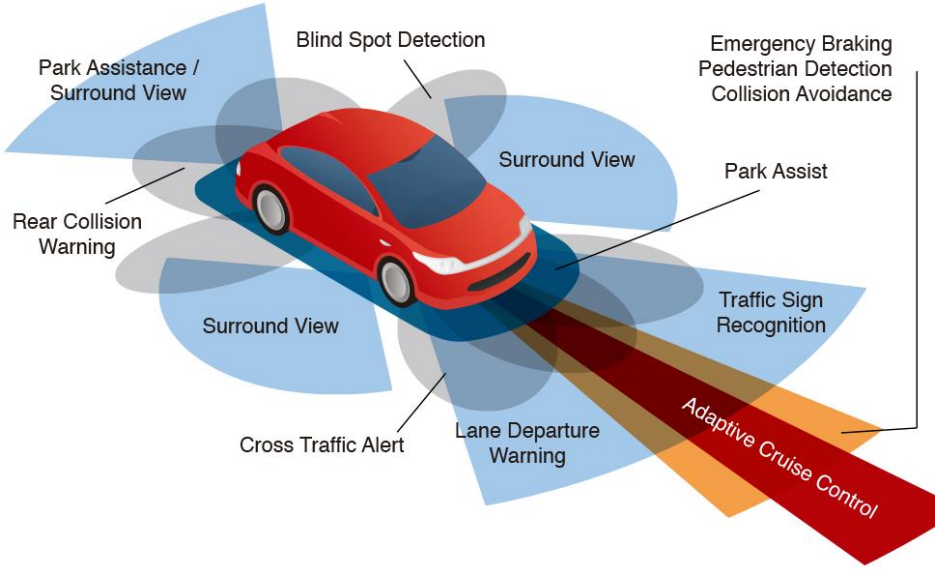
智能制造大数据

- **汽车制造过程中的“所有数据”**
  - 装配产线数据：每个工序的时间，操作工人，零部件，装配设备（如玻璃装配机械臂）的传感器数据
  - 冲压与压铸、焊装、涂装的所有工艺参数，所有传感监测数据
  - 来自零部件供应商的制造数据
  - 下线检测数据
  - 检测后的返修数据

网联汽车大数据

- 每辆车每秒可以产生数 Gigabytes 的数据

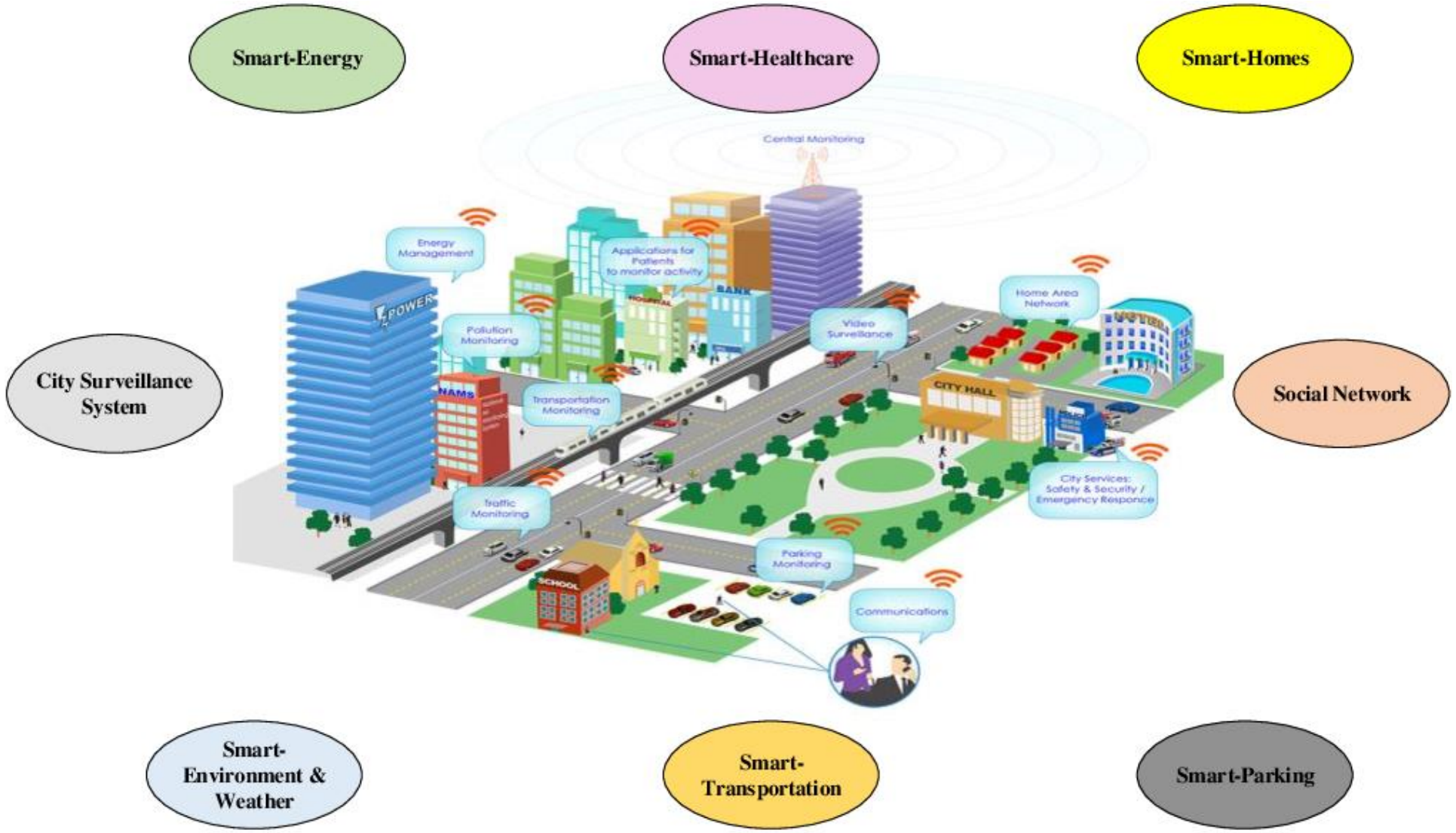




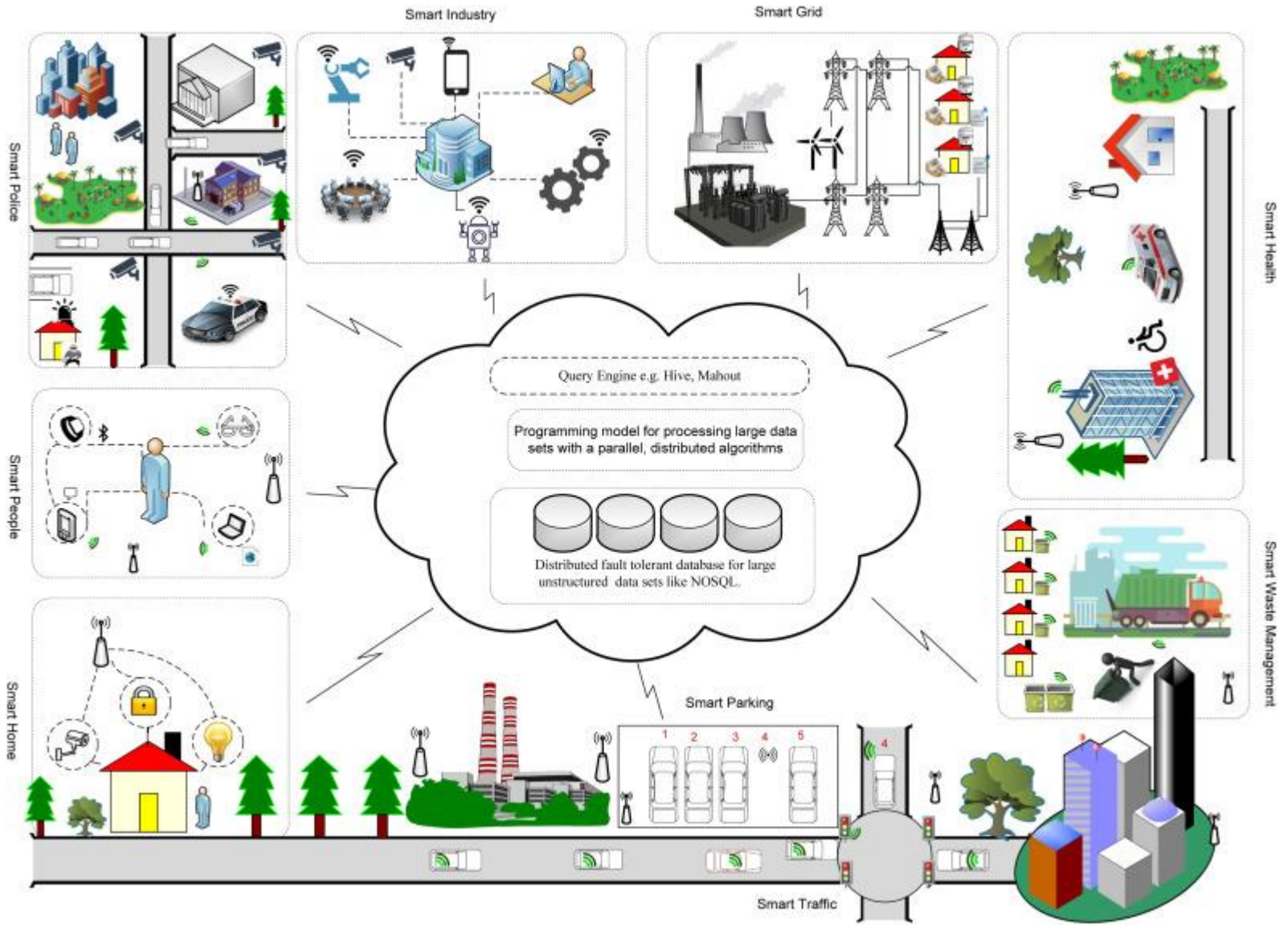
网联汽车大数据

- **每辆车每秒可以产生数 Gigabytes 的数据**
  - 车辆动力部分的传感器，如发动机温度、喷油量、电池温度、电压电流等
  - 车辆底盘部分的传感器，如加速度、角速度、轮速等
  - 超声波传感器、毫米波雷达、倒车影像摄像头
  - 激光雷达、高清摄像头、环视图像
  - 驾驶员操作数据，如方向盘、多媒体、手机app等
  - 车辆电控单元内部处理过的信息，如故障诊断码等
  - 车辆地理位置信息
- **一家车企 100万辆车/年，或一个城市 100 万辆车**

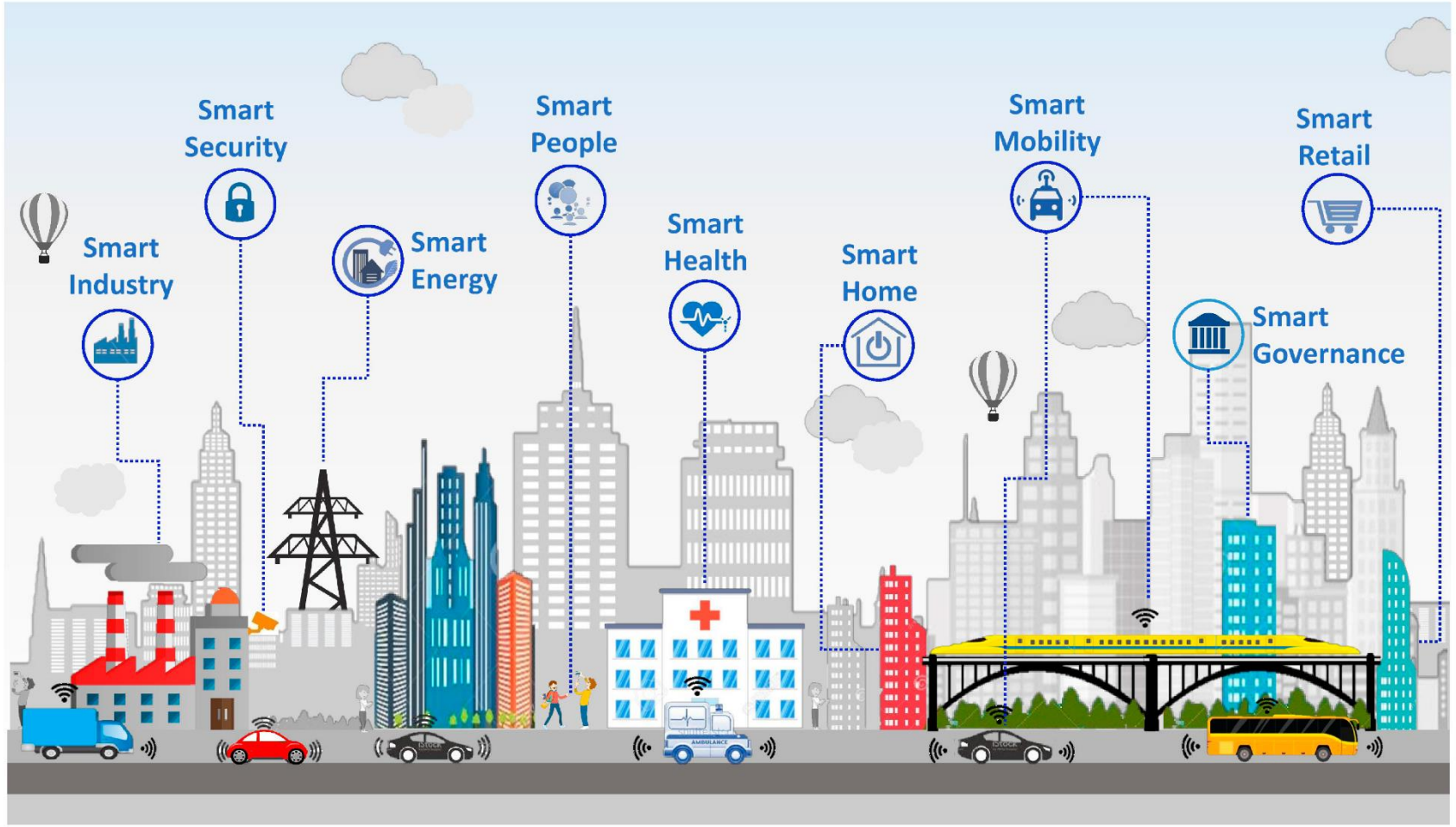
智慧城市大数据


Smart-Energy
Smart-Healthcare
Smart-Homes
City Surveillance System
Social Network
Smart-Environment & Weather
Smart-Transportation
Smart-Parking
Central Monitoring
Energy Management
Applications for Patients to monitor activity
Pollution Monitoring
Video Surveillance
Home Area Network
Transportation Monitoring
Traffic Monitoring
City Services: Safety & Security / Emergency Response
Parking Monitoring
Communications
POWER
HOSPITAL
BANK
NAMs
CITY HALL
SCHOOL

智慧城市大数据


Smart Industry
Smart Grid
Smart Police
Smart Health
Query Engine e.g. Hive, Mahout
Programming model for processing large data sets with a parallel, distributed algorithms
Distributed fault tolerant database for large unstructured data sets like NOSQL.
Smart People
Smart Waste Management
Smart Home
Smart Parking
Smart Traffic

智慧城市大数据


Smart Industry
Smart Security
Smart Energy
Smart People
Smart Health
Smart Home
Smart Mobility
Smart Governance
Smart Retail

大数据分析
(Big Data Analytics)

大数据分析的目标

- 将数据转化为认识和洞察，再转化为行动



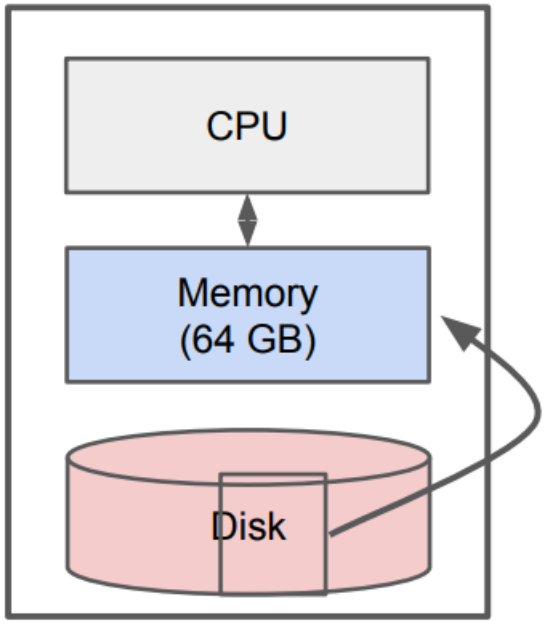
大数据处理的核心思路

- **大数据处理是分布式的**

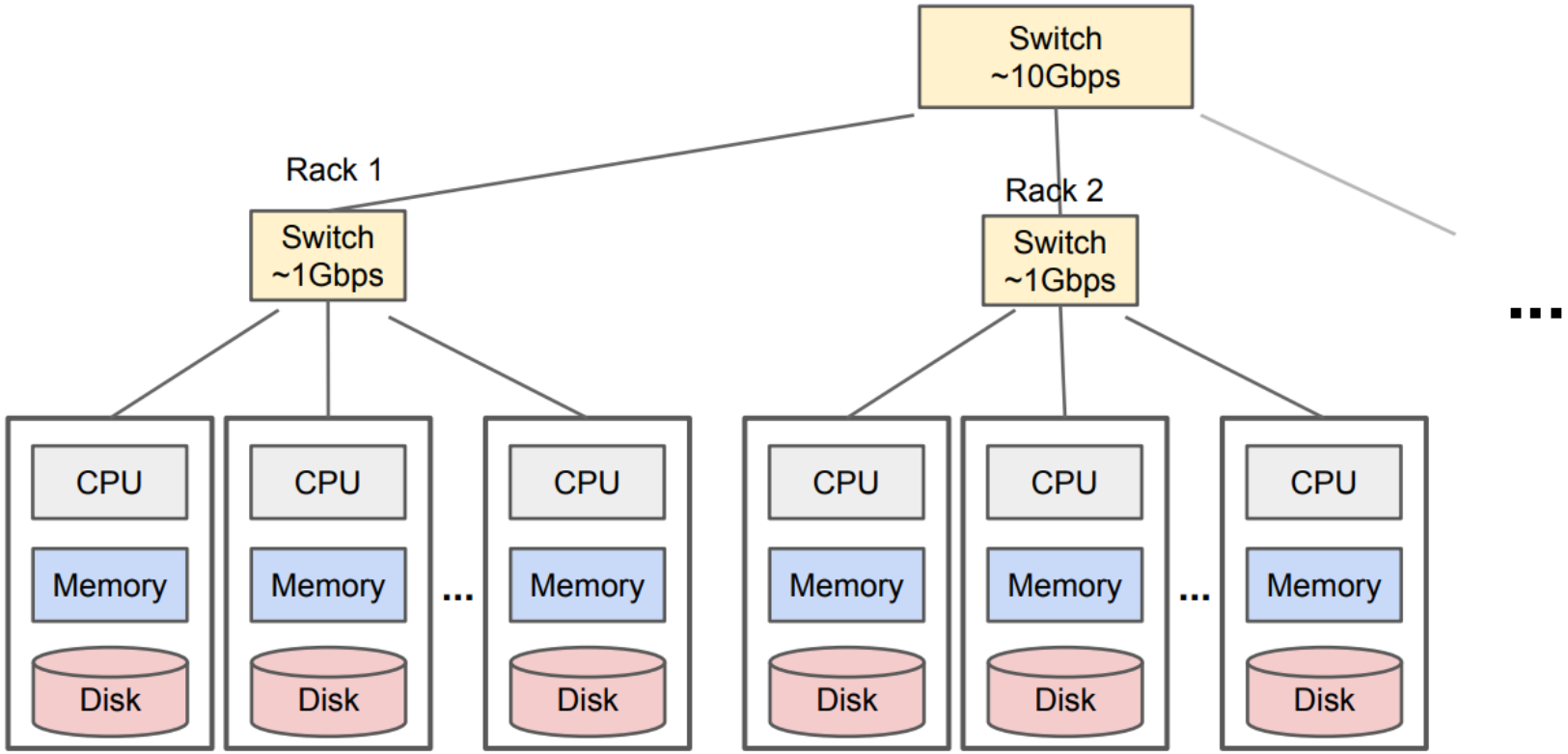
- **因为其计算量需求大**
  - Volume：数据量大
  - Variety：数据类型繁多
  - Velocity：数据速度快

集中式数据处理

- 内存中可以存放下所有数据
- CPU 可以快速地读取数据



分布式数据处理


Switch
~10Gbps
Rack 1
Switch
~1Gbps
Rack 2
Switch
~1Gbps
...
CPU
Memory
Disk
CPU
Memory
Disk
...
CPU
Memory
Disk
CPU
Memory
Disk
CPU
Memory
Disk
...
CPU
Memory
Disk

分布式数据处理

- **节点可能出现故障**
  - 1000个节点，假如每天一个故障

数据冗余储存

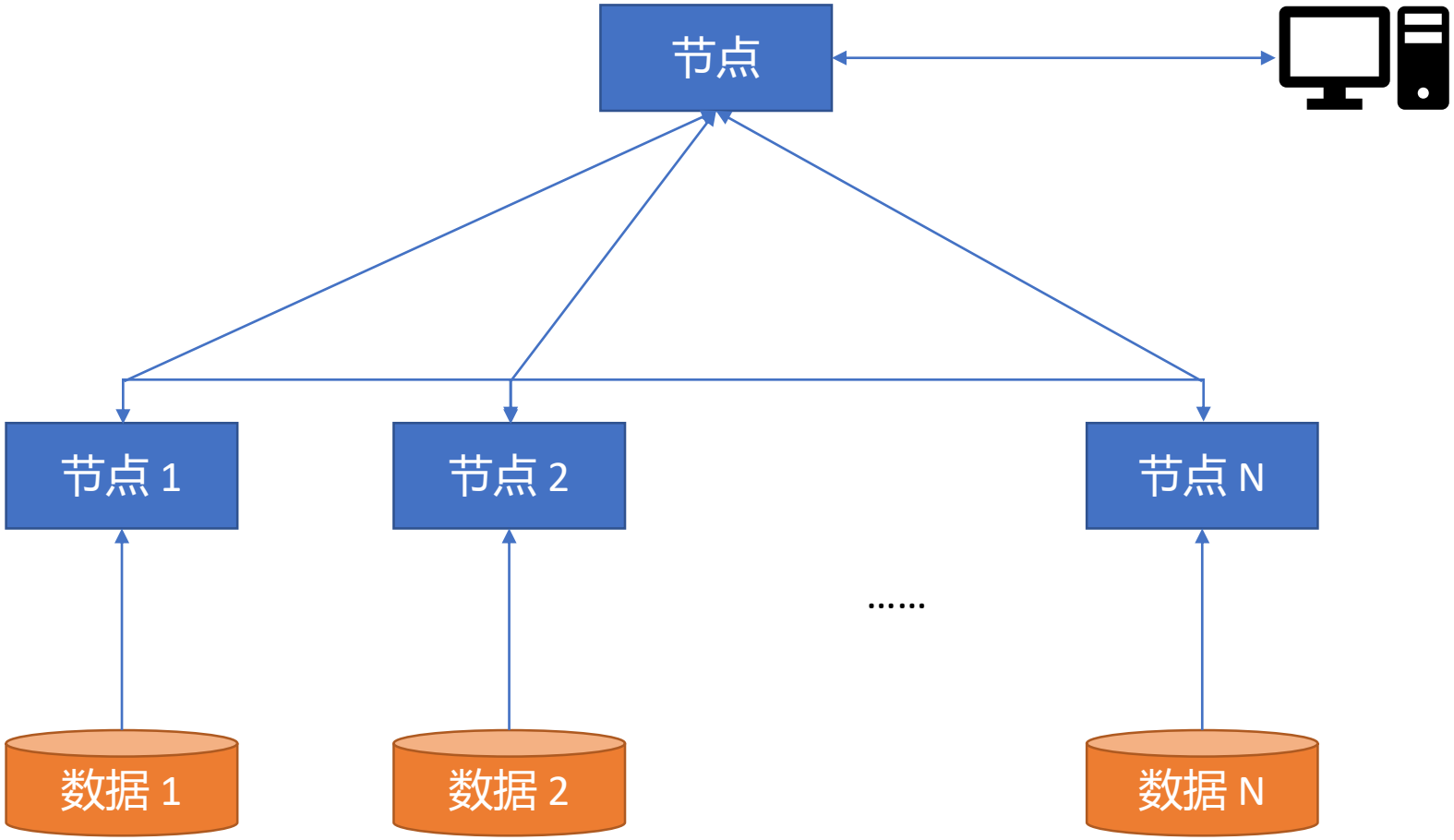
- **网络速度是瓶颈**
  - 1-10 Gb/s

把计算任务搬到数据所在的节点，而不是反过来

- **编程复杂**

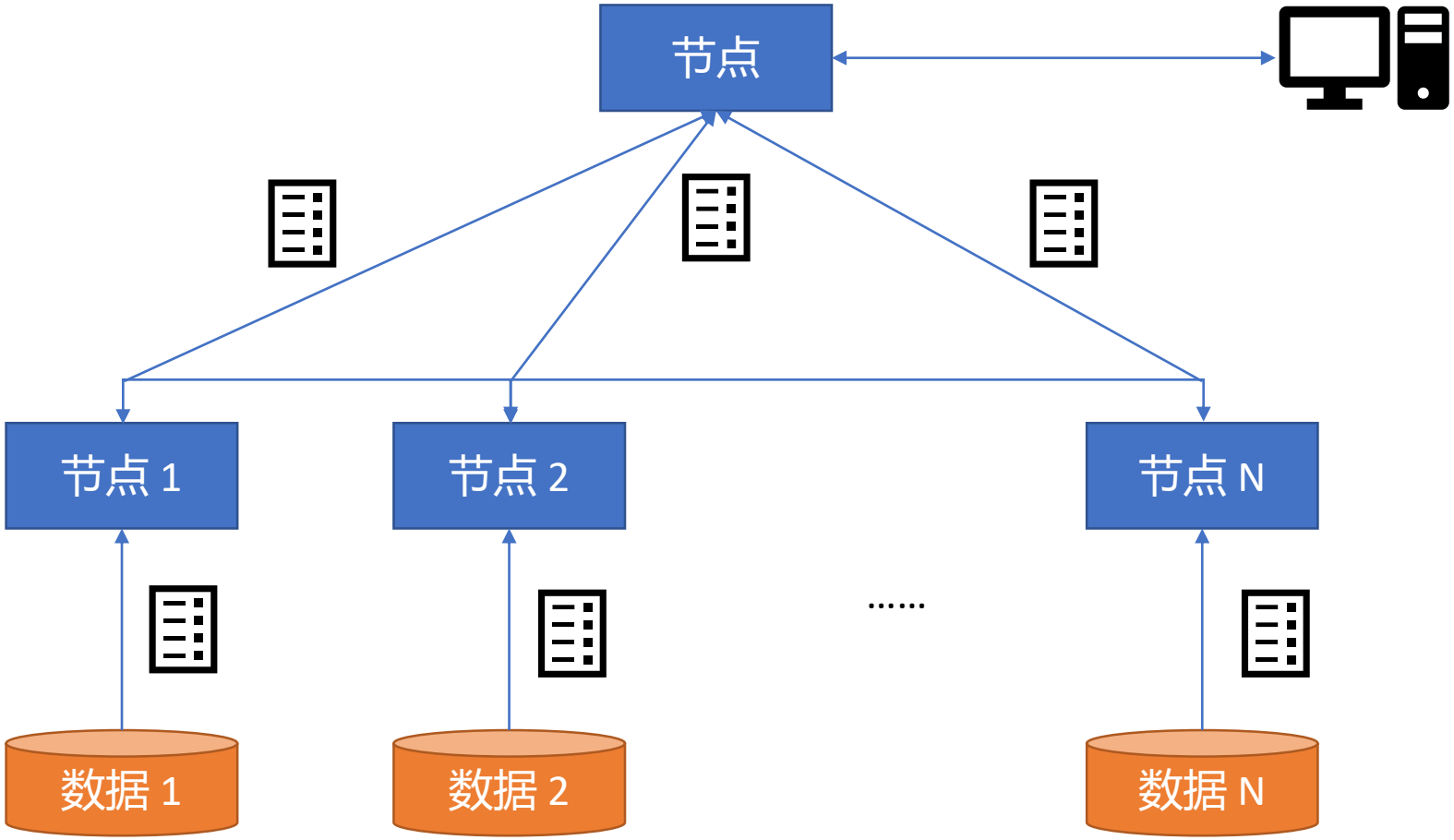
提供易用的工具

分布式数据处理



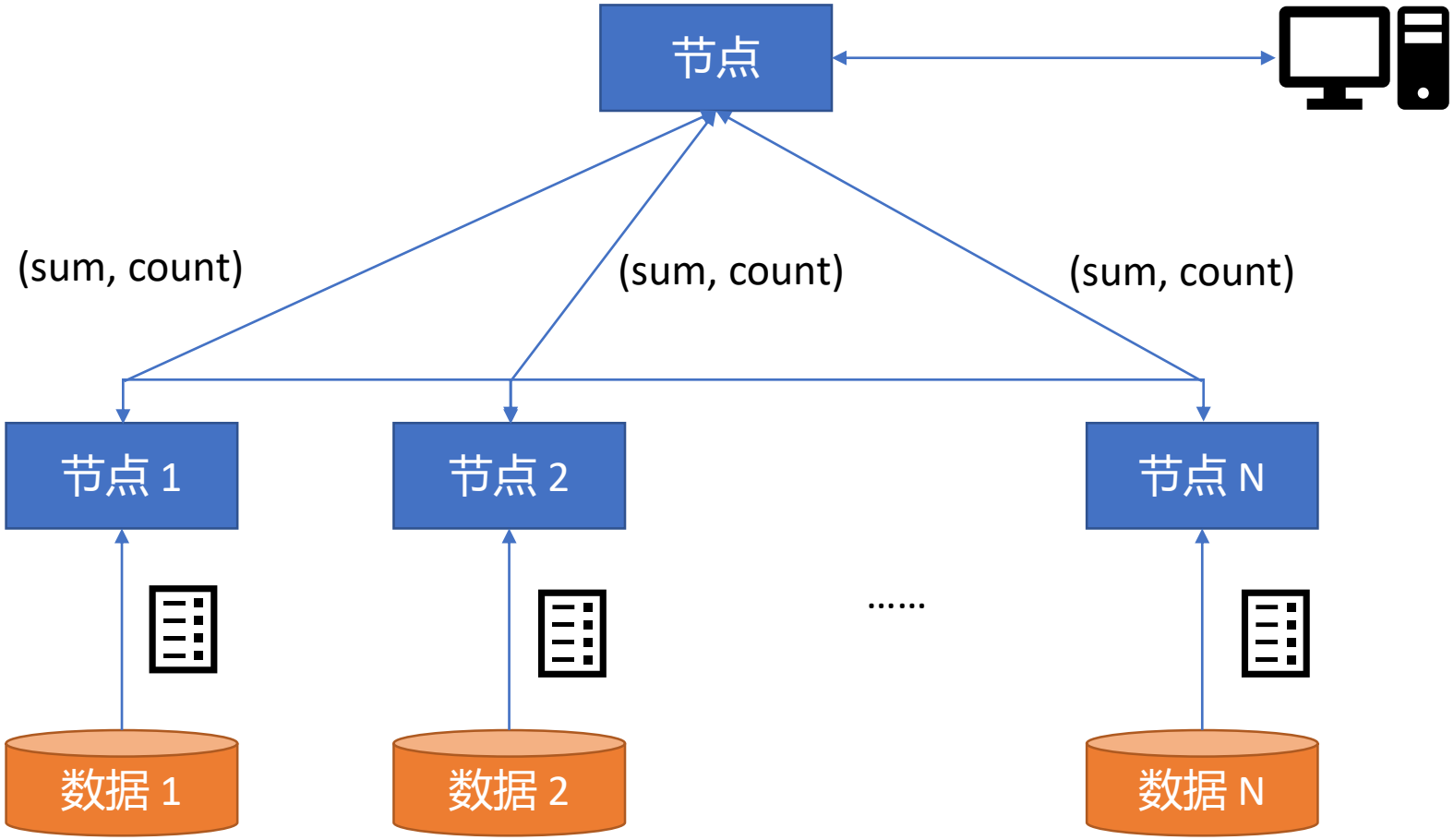
分布式数据处理

- 计算平均值：方法一



分布式数据处理

- 计算平均值：方法二



MapReduce 方法

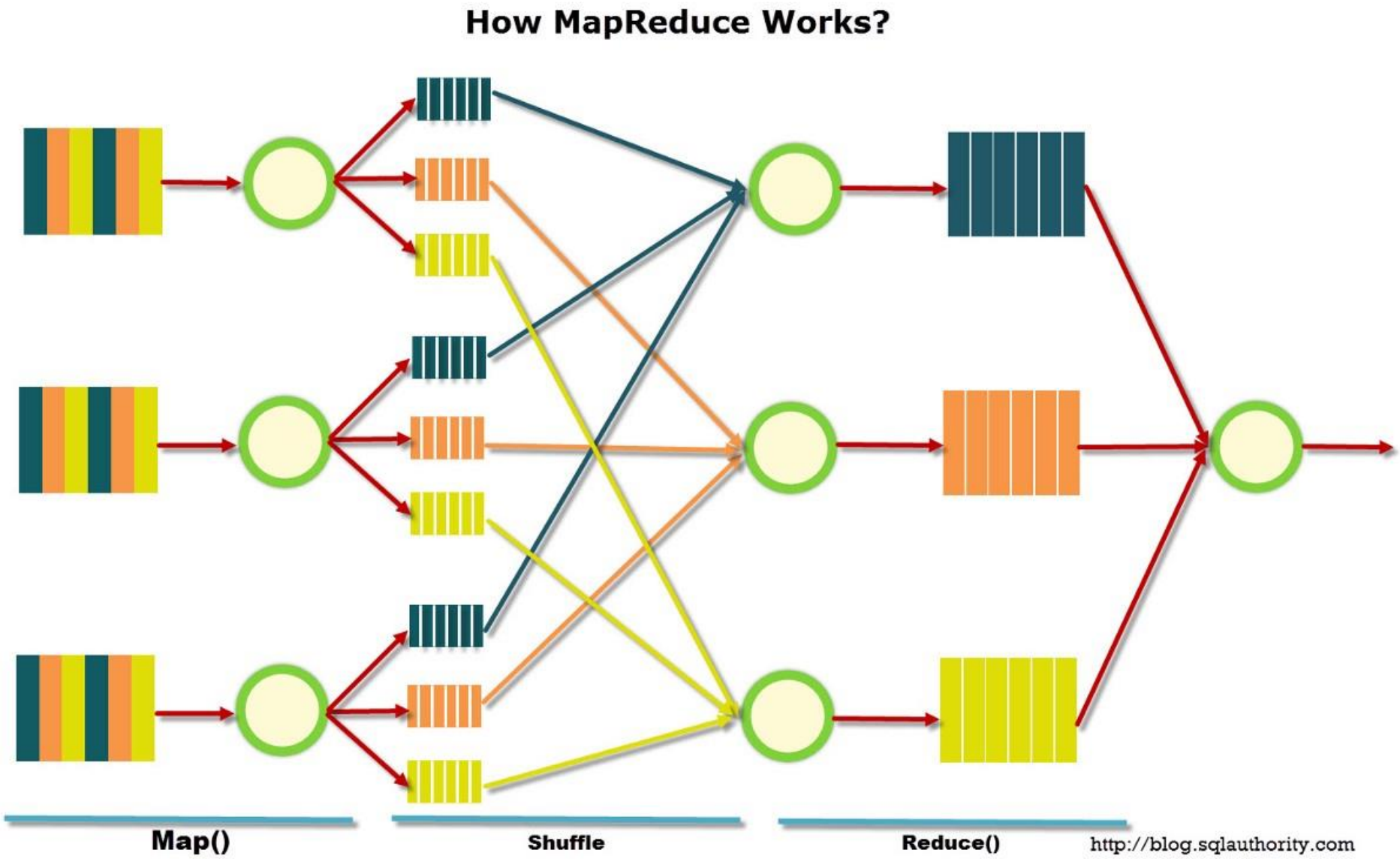
- **MapReduce 是一种分布式并行编程模型，用于大规模数据的并行计算**

- **MapReduce 将复杂的、运行于大规模集群上的并行计算过程高度抽象到两个函数**
  - Map()
  - Reduce()

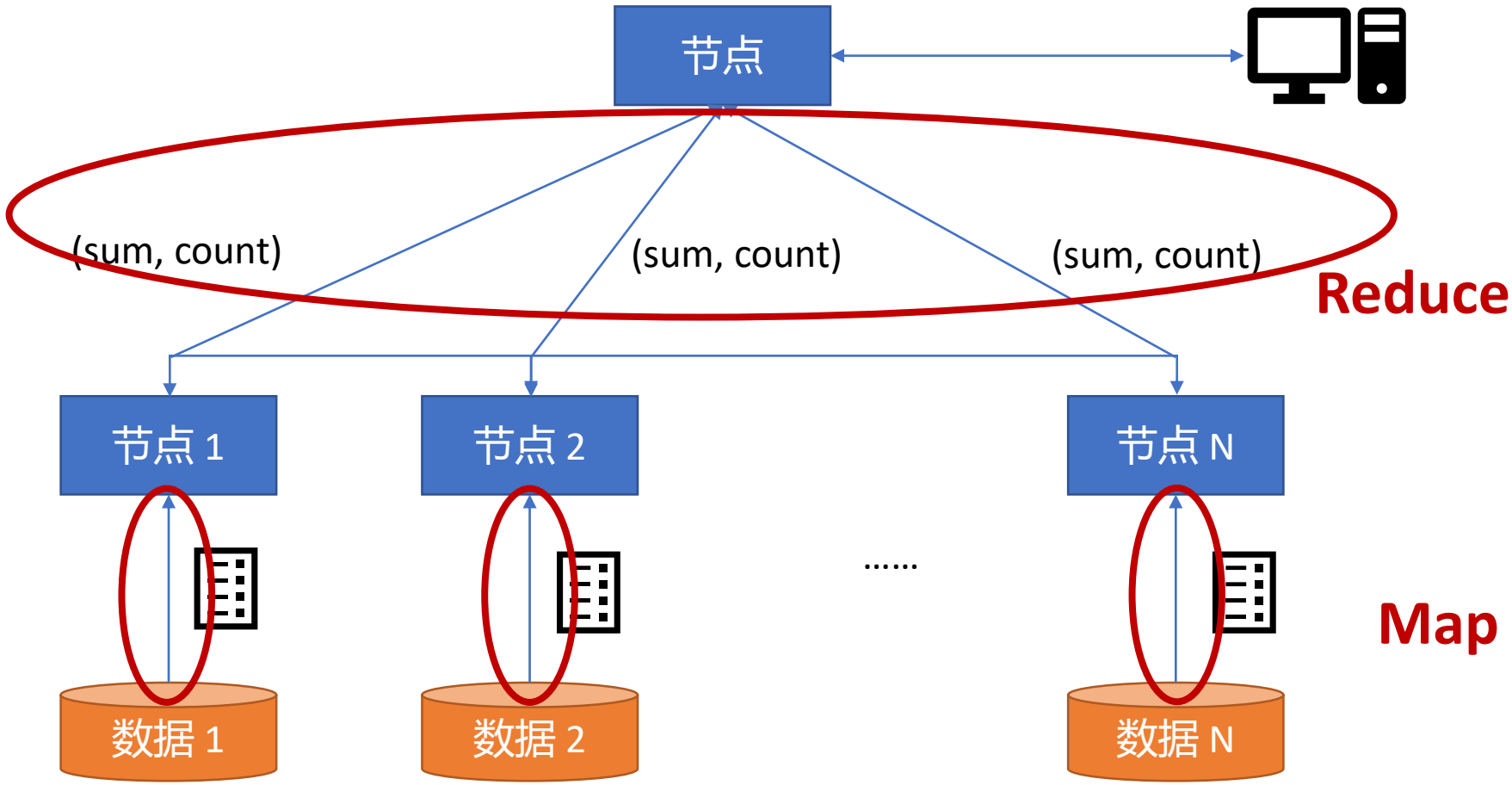
- **MapReduce 极大地方便了分布式编程工作**

MapReduce 方法

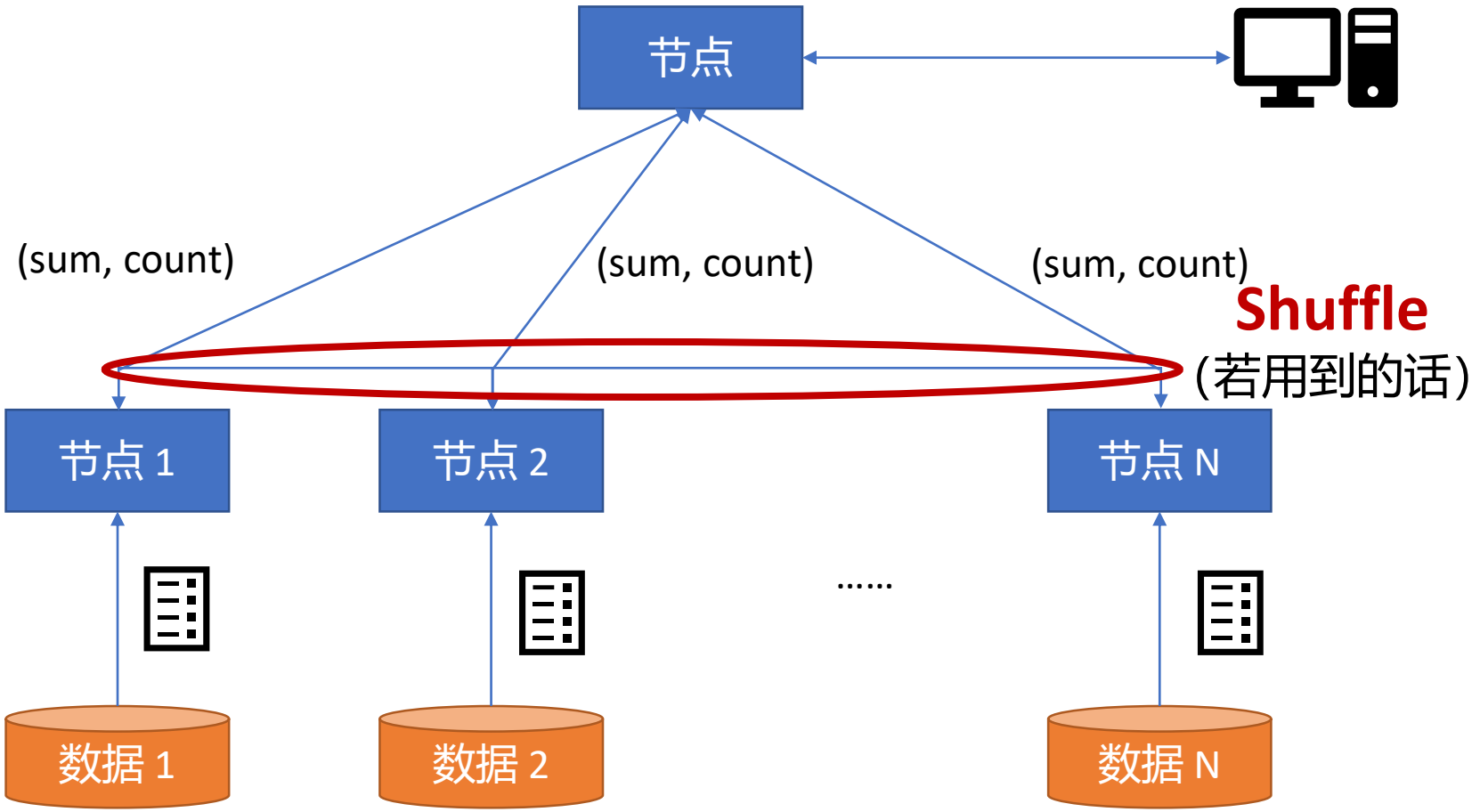
-
-



MapReduce：计算平均值

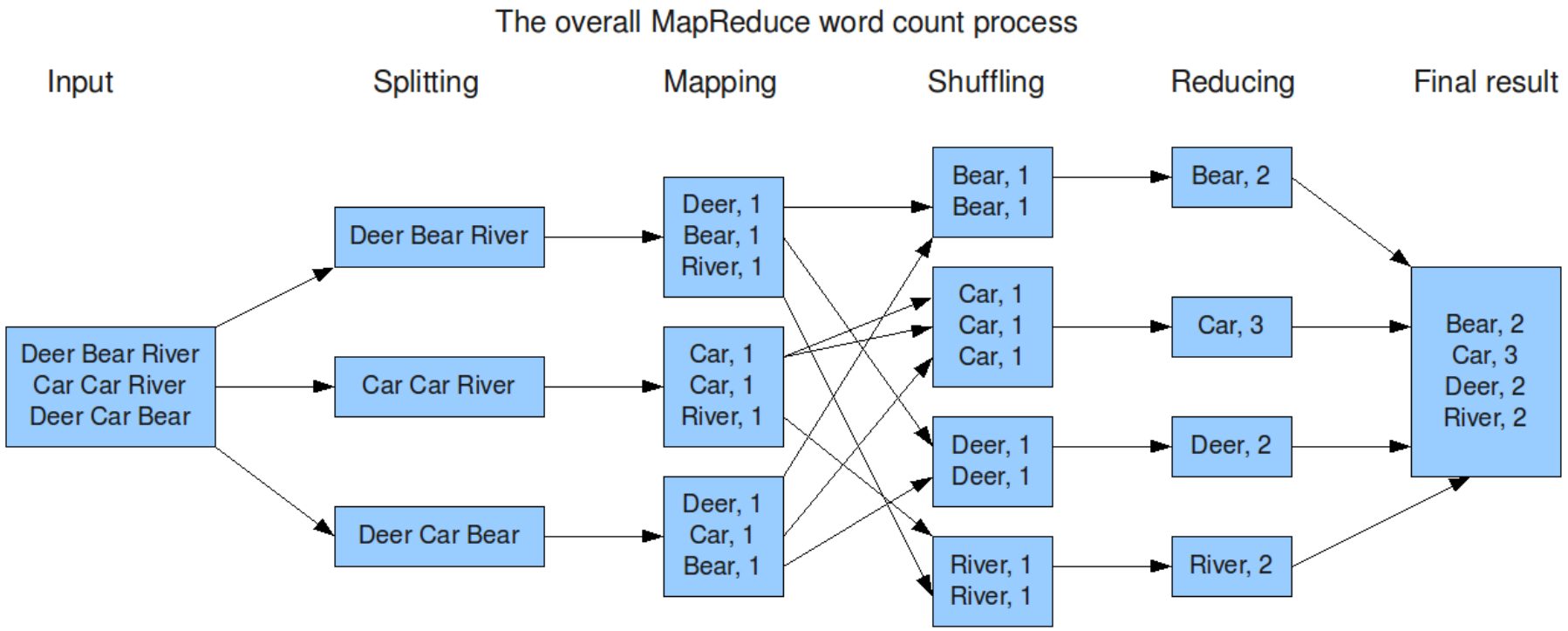


MapReduce：计算平均值



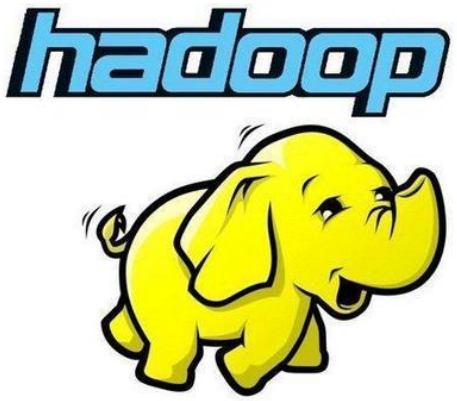
MapReduce 方法

- **计算单词出现次数**



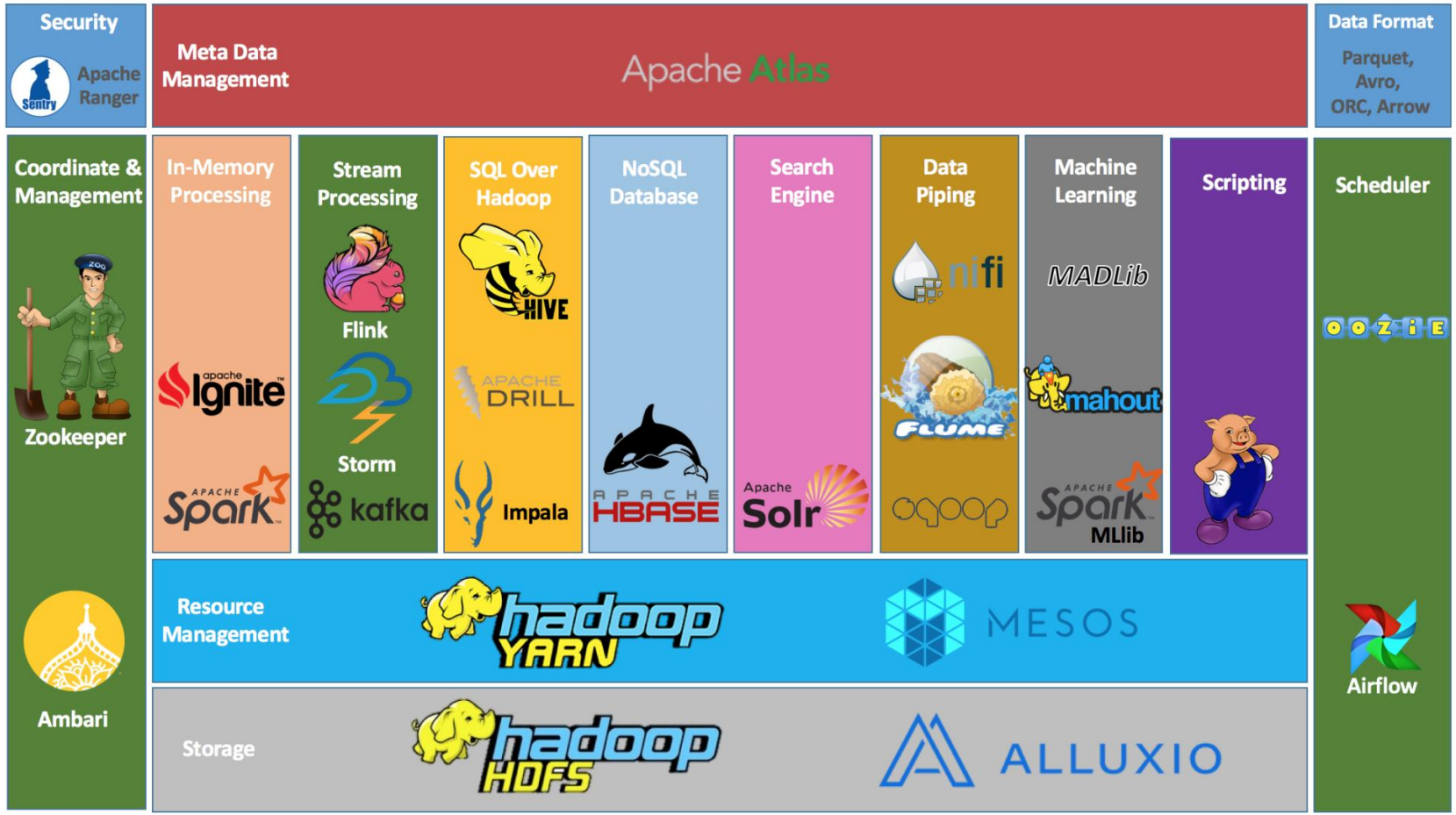
Hadoop

- **Hadoop是开源的分布式计算平台，是基于Java语言开发的，有很好的跨平台特性，并且可以部署在廉价的计算机集群中**



- **Hadoop 平台主要为分布式并行大数据的储存和计算服务**

Hadoop 生态系统

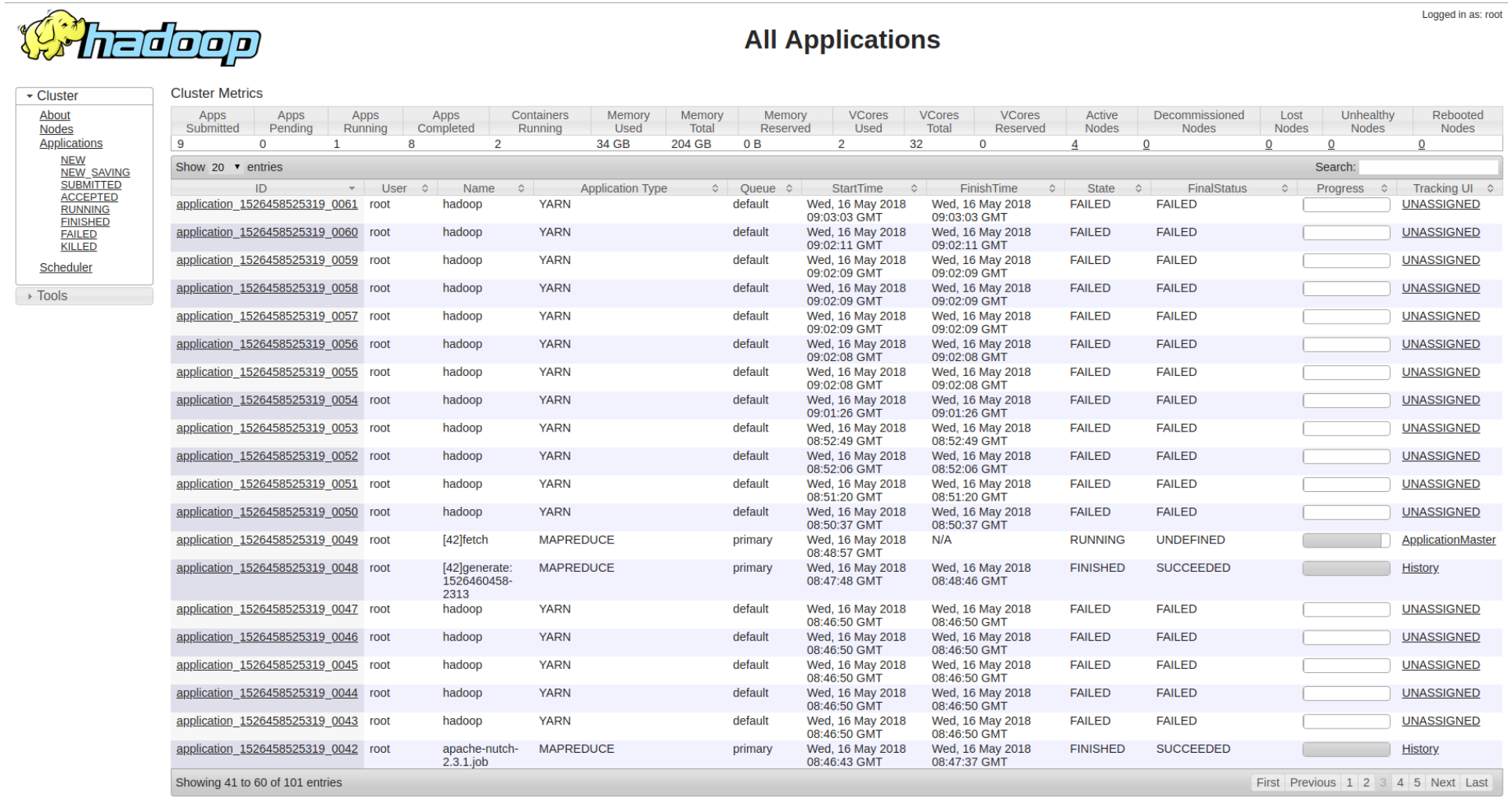
- 
Security
Apache Sentry
Ranger
Meta Data Management
Apache Atlas
Data Format
Parquet, Avro, ORC, Arrow
Coordinate & Management
Zookeeper
Ambari
In-Memory Processing
Apache Ignite
Apache Spark
Stream Processing
Flink
Storm
kafka
SQL Over Hadoop
HIVE
APACHE DRILL
Impala
NoSQL Database
APACHE HBASE
Search Engine
Apache Solr
Data Piping
nifi
FLUME
sqoop
Machine Learning
MADLib
mahout
APACHE Spark MLlib
Scripting
Scheduler
OOZIE
Airflow
Resource Management
hadoop YARN
MESOS
Storage
hadoop HDFS
ALLUXIO

Hadoop 生态系统

- HDFS: Hadoop Distributed File System，文件系统
- YARN: Yet Another Resource Negotiator，资源管理
- MapReduce: Programming based Data Processing
- Spark: In-Memory data processing，内存计算
- PIG, HIVE: Query based processing of data services，表格查询
- HBase: NoSQL Database，数据库
- Mahout, Spark MLLib: Machine Learning libraries，机器学习
- Solar, Lucene: Searching and Indexing，搜索
- Zookeeper: Managing cluster，集群管理
- Oozie: Job Scheduling，作业调度
- Kafka: Real-time big data analytics，实时大数据分析

Hadoop 生态系统：YARN

- **YARN 的 Web UI**



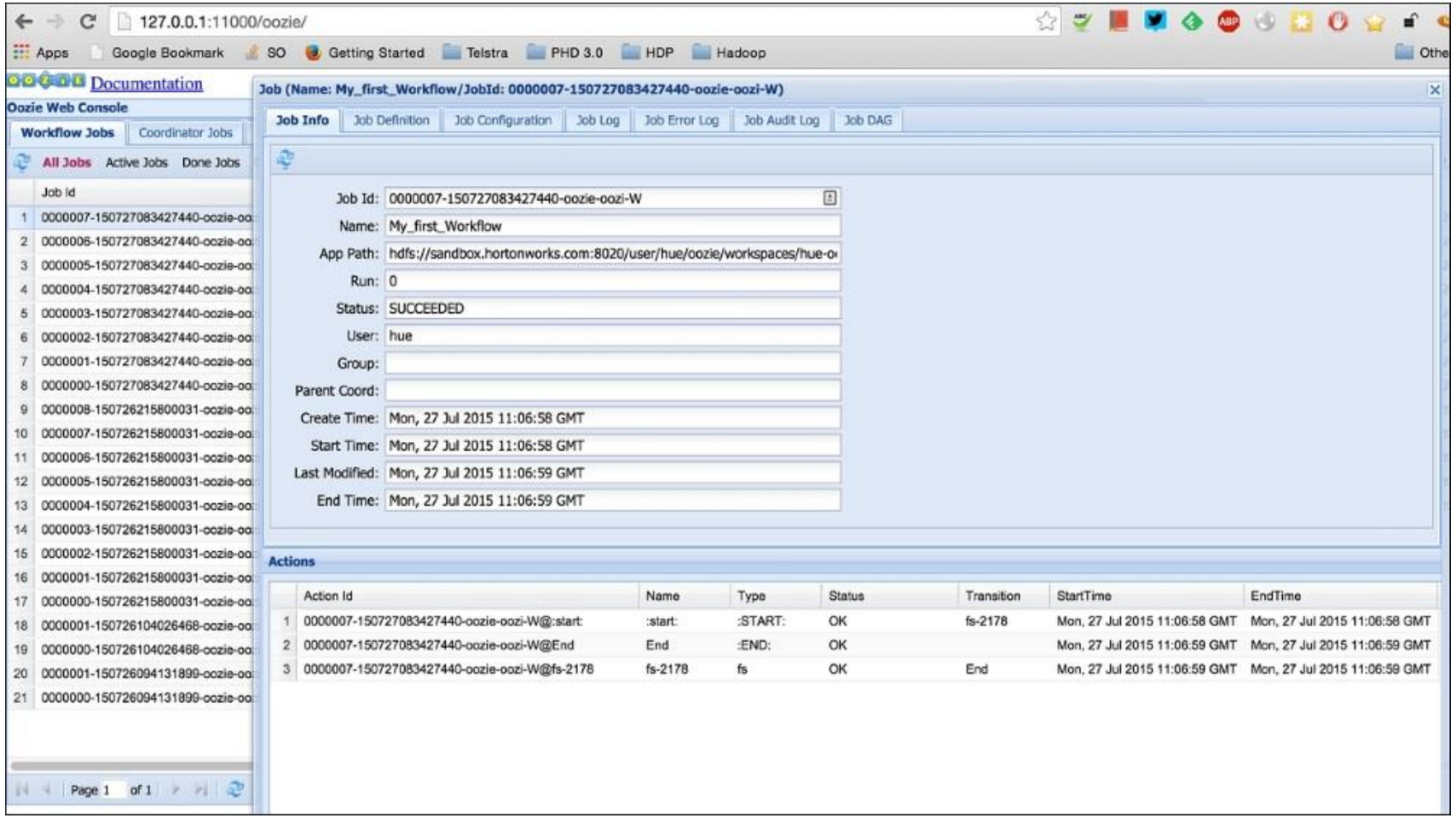
Hadoop 生态系统： HIVE

- **在命令行中使用 HIVE**

```
hive> select f1, sum(f2) from cate group by f1;
Total MapReduce jobs = 1
Launching Job 1 out of 1
Number of reduce tasks not specified. Estimated from input data size: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapred.reduce.tasks=<number>
Starting Job = job_201312102209_0011, Tracking URL = http://localhost:50030/jobd
etails.jsp?jobid=job_201312102209_0011
Kill Command = /usr/lib/hadoop/bin/hadoop job  -Dmapred.job.tracker=localhost:80
21 -kill job_201312102209_0011
2013-12-11 01:05:36,284 Stage-1 map = 0%,  reduce = 0%
2013-12-11 01:05:38,292 Stage-1 map = 100%,  reduce = 0%
2013-12-11 01:05:45,326 Stage-1 map = 100%,  reduce = 100%
Ended Job = job_201312102209_0011
OK
category1       4000
category2       800
category3       10000
Time taken: 12.453 seconds
```

Hadoop 生态系统：Oozie

- **Oozie Web Console**

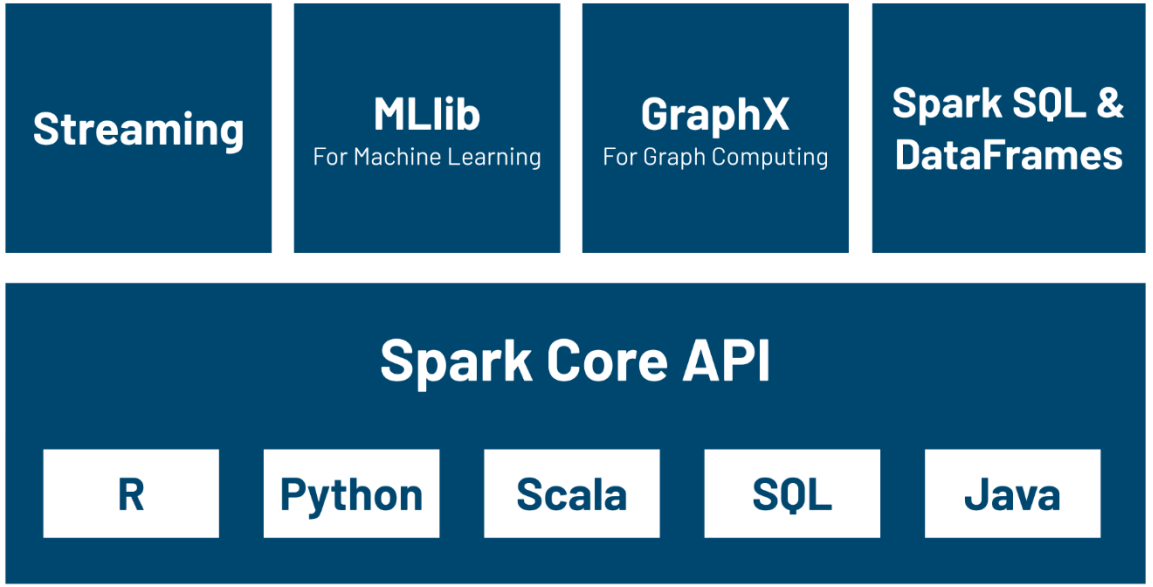


Spark

- **Spark 是一个快速、通用、可扩展的大数据分析引擎**
  - 2009年诞生于加州大学伯克利分校 AMPLab，2010年开源，2013年6月成为 Apache 孵化项目，2014年2月成为 Apache 顶级项目
- **Spark 基于内存运算，速度快**
- **Spark 是很多数据科学家的首选大数据分析工具**

Spark

- **Spark 可处理实时串流大数据（Spark steaming），可进行大数据机器学习（Spark Mllib），可以处理图计算（Spark GraphX）、可以使用类似 SQL 的语言处理表格数据（Spark SQL）**
- **Spark 支持 Scala、Python、Java 等多种编程语言**



- **实际使用时，基于 Spark 提供的 API 编程**

```
from pyspark import SparkContext, SparkConf
```

PySpark requires the same minor version of Python in both driver and workers. It uses the default python version in PATH, you can specify which version of Python you want to use by `PYSPARK_PYTHON`, for example:

```
$ PYSPARK_PYTHON=python3.8 bin/pyspark
$ PYSPARK_PYTHON=/path-to-your-pypy/pypy bin/spark-submit examples/src/main/python/pi.py
```

Initializing Spark

Scala Java Python

The first thing a Spark program must do is to create a SparkContext object, which tells Spark how to access a cluster. To create a `SparkContext` you first need to build a SparkConf object that contains information about your application.

```
conf = SparkConf().setAppName(appName).setMaster(master)
sc = SparkContext(conf=conf)
```

- **实际使用时，基于 Spark 提供的 API 编程**

Basics

Scala Java Python

To illustrate RDD basics, consider the simple program below:

```
lines = sc.textFile("data.txt")
lineLengths = lines.map(lambda s: len(s))
totalLength = lineLengths.reduce(lambda a, b: a + b)
```

The first line defines a base RDD from an external file. This dataset is not loaded in memory or otherwise acted on: `lines` is merely a pointer to the file. The second line defines `lineLengths` as the result of a `map` transformation. Again, `lineLengths` is *not* immediately computed, due to laziness. Finally, we run `reduce`, which is an action. At this point Spark breaks the computation into tasks to run on separate machines, and each machine runs both its part of the map and a local reduction, returning only its answer to the driver program.

If we also wanted to use `lineLengths` again later, we could add:

```
lineLengths.persist()
```

before the `reduce`, which would cause `lineLengths` to be saved in memory after the first time it is computed.

Spark

- **Spark 支持两种运行方式：Spark-submit 和 Spark-shell**
  - Spark-submit：写好代码，提交任务
  - Spark-shell：在 Python Shell 等环境下交互

- **可以使用 Jupyter Notebook，用 Spark-shell 的形式调试 Spark Python 代码**

数据可视化

- 大数据在经过可视化后， 可以更加直观地呈现

- 大部分可视化的数据是大数据在 MapReduce 后的“小数据”



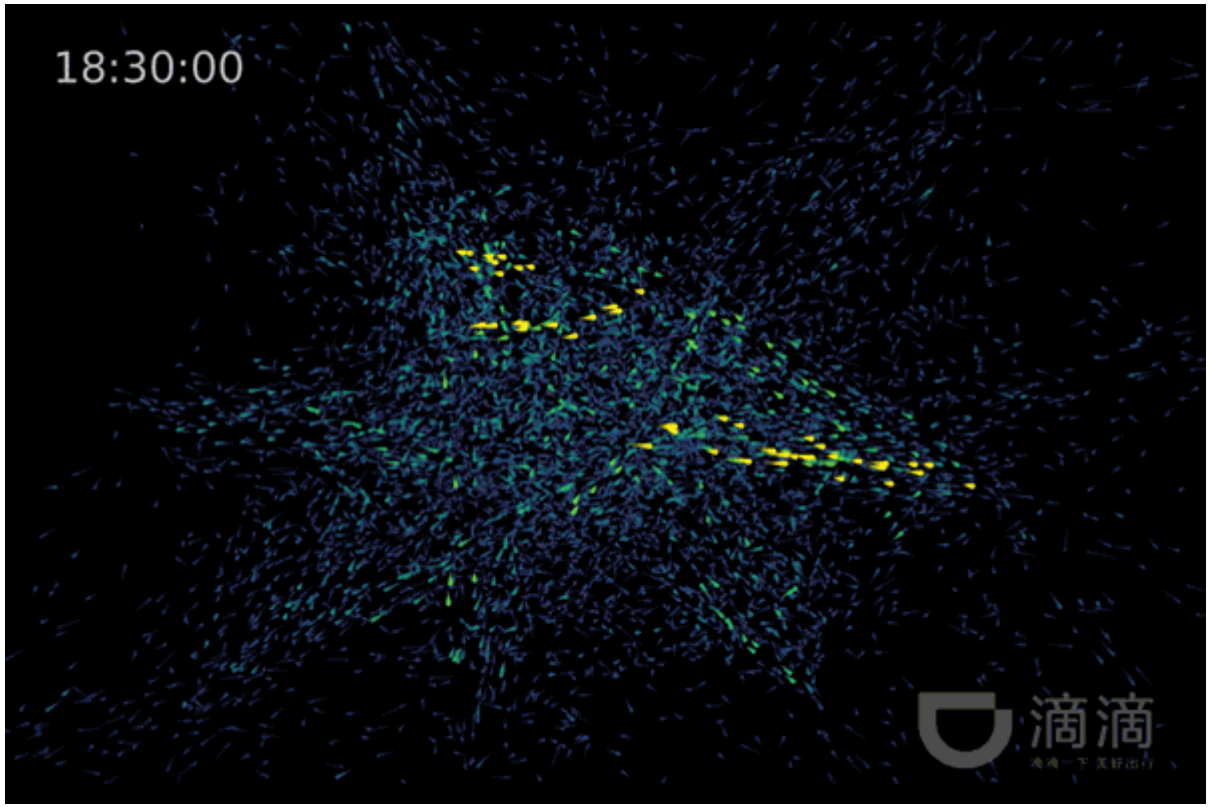
数据可视化

- 天猫双11大屏（大数据 → 小数据）



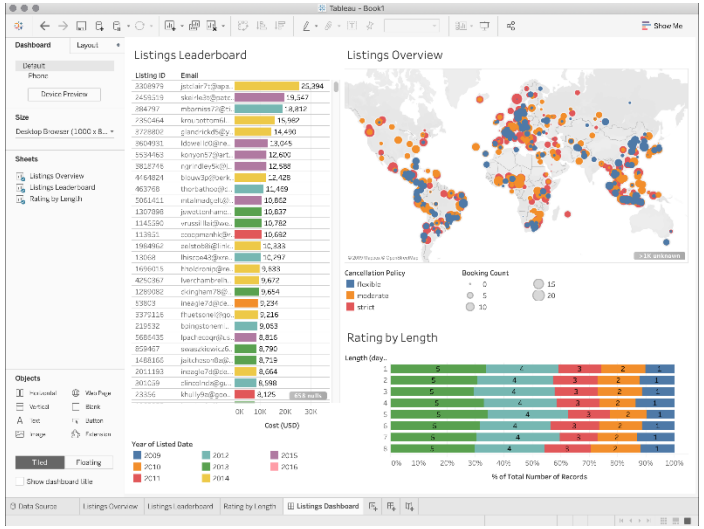
数据可视化

- 滴滴大数据描述的北京早晚高峰通勤（“比较大、但不是特别大”的数据）

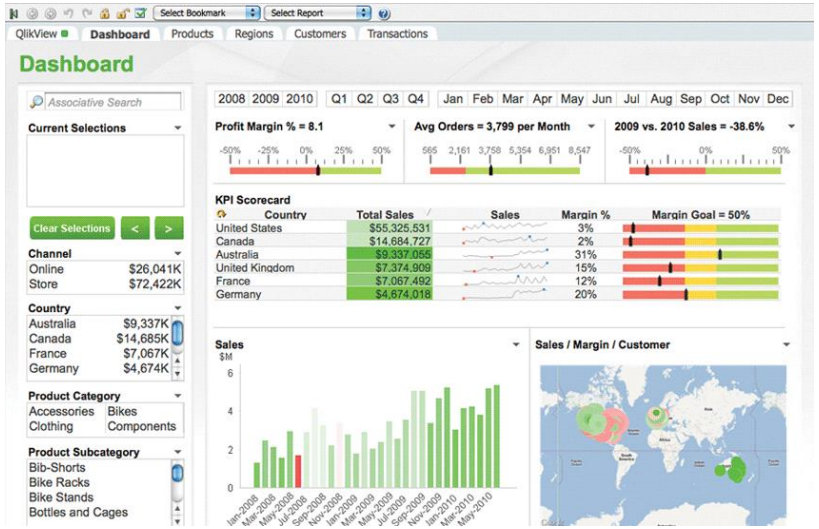


数据可视化

- 可视化既可以通过编程（如利用 Spark 加 Python 中常用的可视化工具）实现，也可以用专门的大数据可视化商业软件实现



Tableua 软件

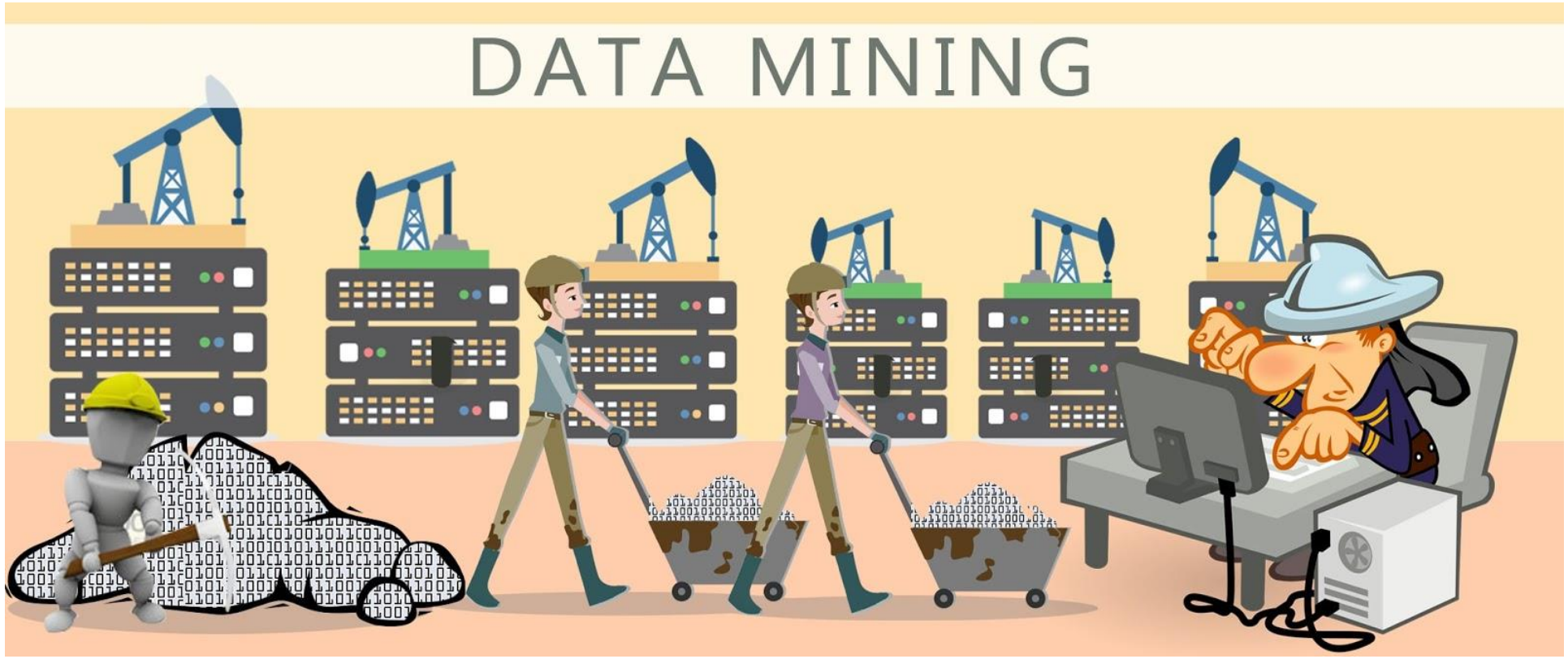


QlikView 软件

数据挖掘

数据挖掘

- 数据挖掘，指的是从大量数据中通过算法搜寻隐藏于其中的信息



可挖掘的数据形式

表 格

CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0.00632	18	2.31	0	0.538	6.575	65.2	4.09	1	296	15.3	396.9	4.98	24
0.02731	0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.9	9.14	21.6
0.02729	0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
0.03237	0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
0.06905	0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.9	5.33	36.2
0.02985	0	2.18	0	0.458	6.43	58.7	6.0622	3	222	18.7	394.12	5.21	28.7
0.08829	12.5	7.87	0	0.524	6.012	66.6	5.5605	5	311	15.2	395.6	12.43	22.9
:	:	:	:	:	:	:	:	:	:	:	:	:	:
:	:	:	:	:	:	:	:	:	:	:	:	:	:

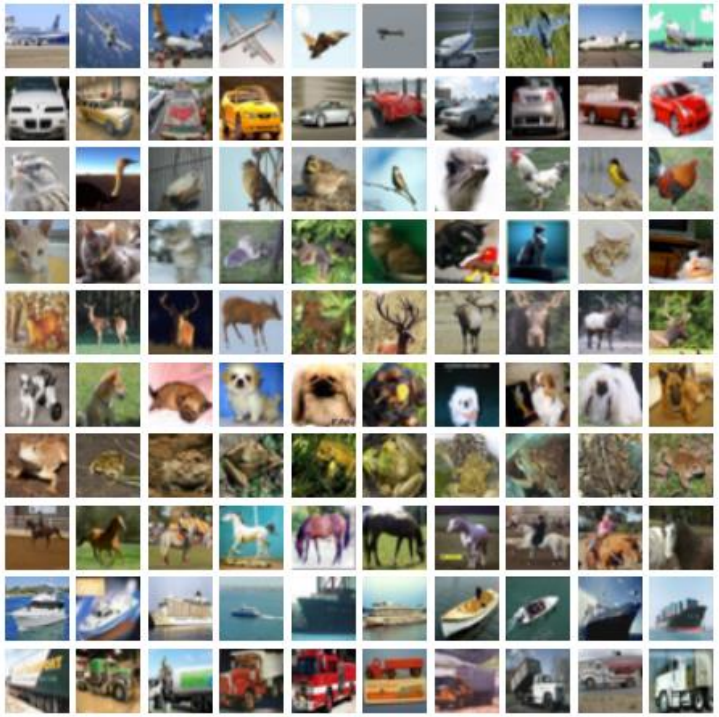
集 合

ID	Items
1	面包，可乐，牛奶
2	啤酒，面包
3	啤酒，可乐，牛奶，婴儿尿布
4	啤酒，面包，婴儿尿布



可挖掘的数据形式

图 片



文 字

V 90. letech 20. století, s širokým uplatněním databázových systémů a rychlým rozvojem síťových technologií, vstoupila databázová technologie také do nové fáze, tedy od správy některých jednoduchých dat v minulosti až po správu různých typů složitých dat, jako je grafika, obrázky, audio, video, elektronické archivy, webové stránky generované různými počítači, a množství dat se také zvyšuje. Zatímco databáze nám poskytuje bohaté informace, odráží také zřejmé charakteristiky masivních informací. V éře informační exploze přináší masivní informace lidem mnoho negativních dopadů, nejdůležitější je, že efektivní informace je obtížné zpřesnit, příliš mnoho zbytečných informací nevyhnutelně vytvoří informační vzdálenost (informační stav přenosové vzdálenosti, je měření překážek, které se vyskytují při přenosu informačního stavu věci, označované jako DIST nebo DIT) a ztráta užitečných znalostí. To je to, co John Nalsbert nazval dilematem "informačně bohatí a chudí na znalosti".

可挖掘的数据形式

序 列

SYNTENIC ASSEMBLIES FOR CG15386

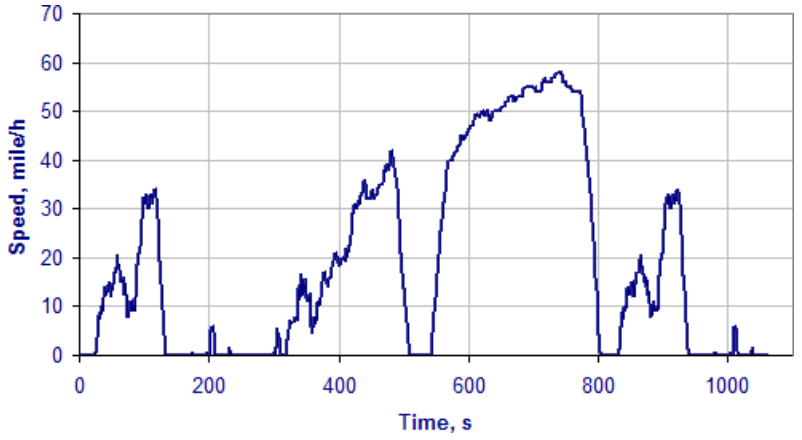
```
MD106   ATGCTTAGTAATCCCTACTTTAAGTCCGTTTTGTGGCTGATTGGCTTCGGAGGAATGGG
NEWC    ATGCTTAGTAATCCCTACTTTAAATCCGTTTTGTGGCTGATTGGCTTCGGAGGAATGGG
W501    ATGCTTAGTAATCCCTACTTTAAGTCCGTTTTGTGGCTGATTGGCTTCGGAGGAATGGG
MD199   ATGCTTAGTAATCCCTACTTTAAGTCCGTTTTGTGGCTGATTGGCTTCGGAGGAATGGG
C1674   ATGCTTAGTAATCCCTACTTTAAGTCCGTTTTGTGGCTGATTGGCTTCGGAGGAATGGG
SIM4    ATGCTTAGTAATCCCTACTTTAAGTCCGTTTTGTGGCTGATTGGCTTCGGAGGAATGGG

MD106   CTACGGCCTAATGGTGCTAACAGAGCCGAACGTCGACAAAATAGAGCGCATCAAAGCCT
NEWC    CTACGGCCTAATGGTGCTAACCGAGCCGAACGTCGACAAAATAGAGCGCATCAAAGCCT
W501    CTACGGCCTAATGGTGCTAACCGAGCCGAACGTCGACAAAATAGAGCGCATCAAAGCCT
MD199   CTACGGCCTAATGGTGCTAACCGAGCCGAACGTCGACAAAATAGAGCGCATCAAAGCCT
C1674   CTACGGCCTAATGGTGCTAACCGAGCCGAACGTCGACAAAATAGAGCGCATCAAAGCCT
SIM4    CTACGGCCTAATGGTGCTAACCGAGCCGAACGTCGACAAAATAGAGCGCATCAAAGCCT

MD106   CCGTTTCAAGTACCAAACTGAGTGCGGATGAGCAGCGAAAGGCTCTGTTTATGAAGAAG
NEWC    CCGTTTCAAGTACCAAACTGAGTGCGGATGAGCAGCGAAAGGCTCTGTTTATGAAGAAG
W501    CCGTTTCAAGTACCAAACTGAGTGCGGATGAGCAGCGAAAGGCTCTGTTTATGAAGAAG
MD199   CCGTTTCAAGTACCAAACTGAGTGCGGATGAGCAGCGAAAGGCTCTGTTTATGAAGAAG
C1674   CCGTTTCAAGTACCAAACTGAGTGCGGATGAGCAGCGAAAGGCTCTGTTTATGAAGAAG
SIM4    CCGTTTCAAGTACCAAACTGAGTGCGGATGAGCAGCGAAAGGCTCTGTTTATGAAGAAG

MD106   CTGCAGGAGGCGTCCACCACCAGTGCCCCAATCTACAGGTCAGCGGCCGAGAAATAG
NEWC    CTGCAGGAGGCGTCCACCACCAGTGCCCCAATCTACAGGTCATCGGCCGAGAAATAG
W501    CTGCAGGAGGCGTCCACCACCACTGCCCCAATCTACAGGTCATCGGCCGAGAAATAG
MD199   CTGCAGGAGGCGTCCACCACCAGTGCCCCAATCTACAGGTCAGCGGCCGAGAAATAG
C1674   CTGCAGGAGGCGTCCACCACCAGTGCCCCAATCTACAGGTCAGCGGCCGAGAAATAG
SIM4    CTGCAGGAGGCGTCCACCACCAGTGCCCCAATCTACAGGTCAGCGGCCGAGAAATAG
```

时序数据



可挖掘的数据形式

空间数据

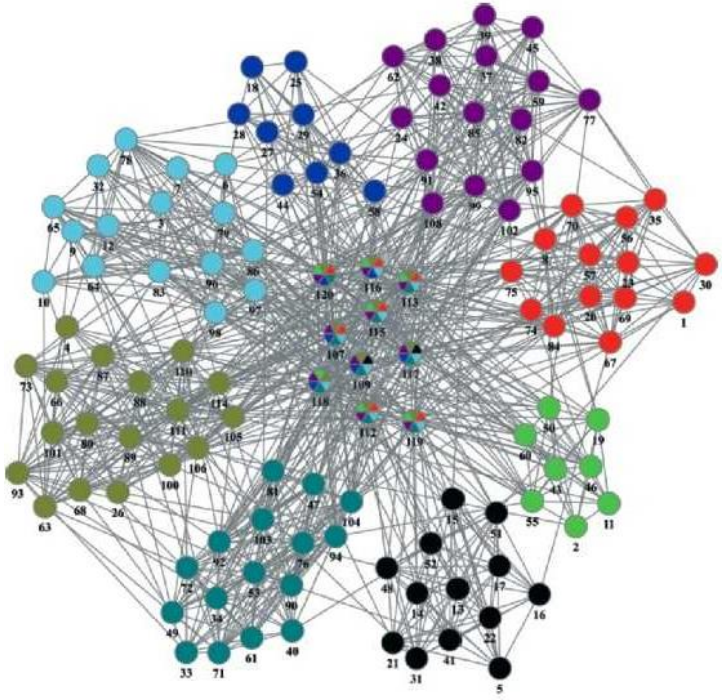


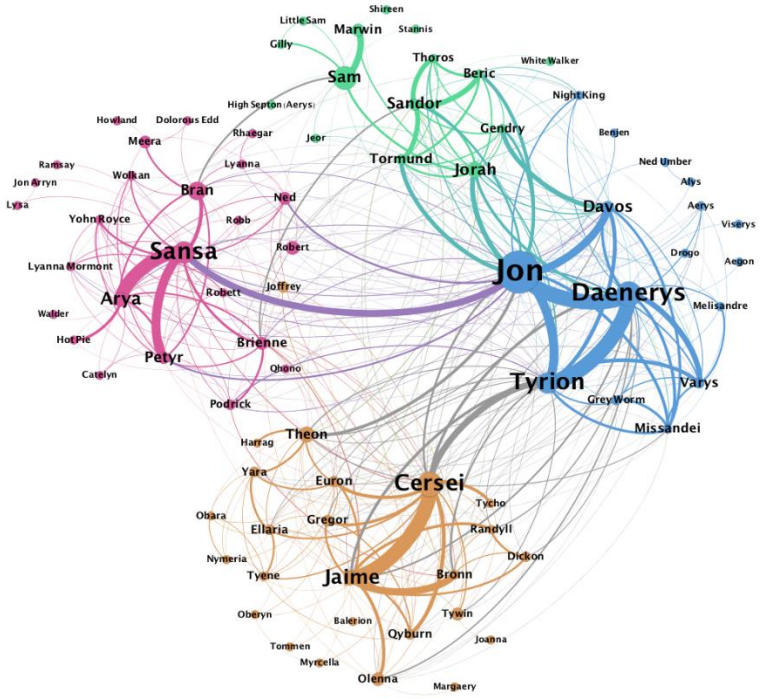
时空数据



可挖掘的数据形式

图（graph）/网络





数据挖掘任务：关联模式挖掘

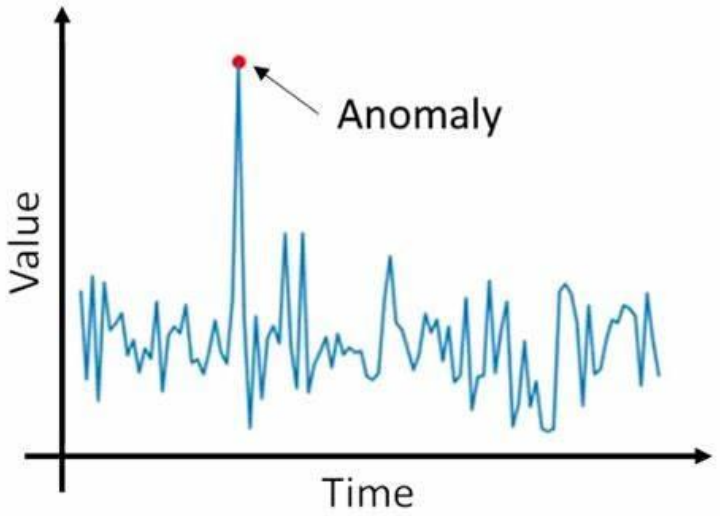
- 关联模式（association pattern）

- 频繁重复出现的项目



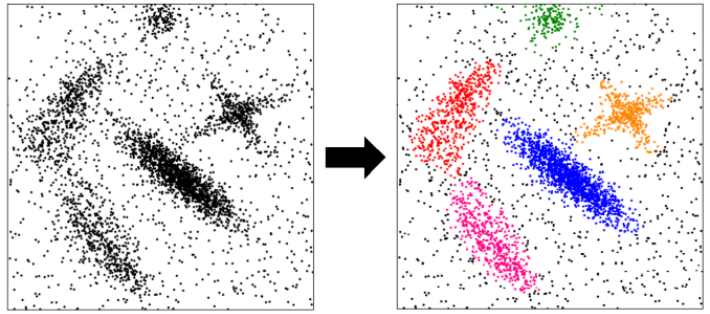
数据挖掘任务：异常检测

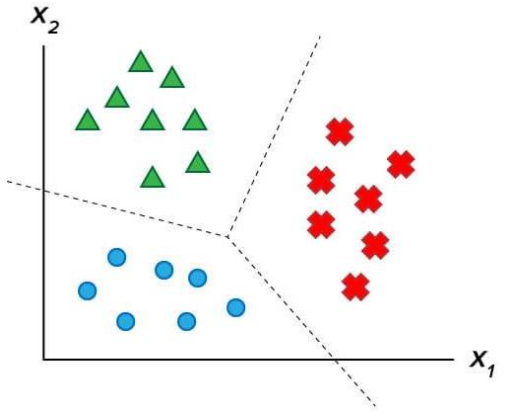
- **从数据中找出异常的部分**
  - 异常的原因：机理性的，随机性的，测量引起的



数据挖掘任务：聚类、分类和预测

- 我们在本课程机器学习部分介绍过了聚类、分类和预测的基本概念和方法





不同数据类型下的挖掘任务

- Table 1.2: Some examples of variation in problem definition with data type

Problem	Time series	Spatial	Sequence	Networks
Patterns	Motif-mining Periodic pattern	Colocation patterns	Sequential patterns Periodic Sequence	Structural patterns
	Trajectory patterns			
Clustering	Shape clusters	Spatial clusters	Sequence clusters	Community detection
	Trajectory clusters			
Outliers	Position outlier Shape outlier	Position outlier Shape outlier	Position outlier Combination outlier	Node outlier Linkage outlier Community outliers
	Trajectory outliers			
Classification	Position classification Shape classification	Position classification Shape classification	Position classification Sequence classification	Collective classification Graph classification
	Trajectory classification			

Aggarwal, Charu C. *Data mining: the textbook*. Vol. 1. New York: springer, 2015.

关联模式

关联模式挖掘的应用场景

- 购物车物品
- 基于数据的用户画像（购物车，音乐歌单，……）
- 故障溯源

关联模式挖掘的例子

- **一个经典的例子，就是 {啤酒，婴儿尿布} 这个意外的组合**

  - 在美国有婴儿的家庭中，一般是母亲在家中照看婴儿，年轻的父亲前去超市购买尿布
  - 父亲在购买尿布的同时，往往会顺便为自己购买啤酒
  - 另一方面，未婚或没有小孩的年轻男性，通常更多地去酒吧喝酒，而不在超市买啤酒
  - 这样就会出现啤酒与尿布这两件看上去不相干的商品经常会出现在同一个购物篮的现象

关联模式挖掘的应用场景

- **如果通过分析超市的购物篮，我们发现买了 X 商品的顾客通常也同时买了 Y，那么我们可以**
  - 把仅购买了 X 的顾客，视为 Y 商品的宣传对象
    - “购买此商品的顾客也同时购买了……”
    - 把 X 和 Y 商品放在同一货架上
  - 仅对 X 商品打折并宣传折扣信息，吸引顾客来买 X 和原价的 Y
  - 设计融合了 X 和 Y 的新商品，比如具有 Y 口味的 X

关联模式挖掘的应用场景

- 我们把**买了 X 商品的顾客通常也同时买了 Y** 记作

$$X \rightarrow Y$$

我们把 $X \rightarrow Y$ （X implies Y）看作一条“规则”，我们通过关联模式分析，来判断这条规则的普适程度

关联模式挖掘

- 我们用 T_i 代表用户 i 的购物篮，一个购物篮是多件项目的集合
- 项集（itemset）：项（item）的集合

  购物篮中一件商品就是一项，若干项的集合就是一个项集。比如 {啤酒，尿布} 构成一个含有两项的项集

- 一个购物篮也是一个项集；某个购物篮项集可能包含 {啤酒，尿布} 这个项集，也可能不包含 {啤酒，尿布} 这个项集

关联模式挖掘

- 支持度（support）：在购物篮数据集 $T_1, T_2, ...$ 中，包含了项集 X 的购物篮的比例

  支持度表明有多少比例的人同时买了项集 X 里的所有商品

- 如果把购物篮里包含项集 X 看作一个事件，将其概率记为 $\mathbb{P}(X)$

$$support(X) = \mathbb{P}(X)$$

关联模式挖掘

- 我们也常常会关心多个项集的支持度，这时的支持度指的是同时包含这些项集的比例。比如在考虑规则 $X \rightarrow Y$是，我们有时也写作

$$Support(X \rightarrow Y) = \mathbb{P}(\text{同时包含X和Y})$$

商品	支持度
啤酒	7.77%
汽水	17.44%
浆果	3.32%
男士护肤品	0.46%

关联规则	支持度
啤酒 → 汽水	1.38%
啤酒 → 浆果	0.08%
啤酒 → 男士护肤品	0.09%

关联模式挖掘

- 置信度（Confidence）：在购物篮数据集 $T_1, T_2, \ldots$ 中，当项集 X 出现时，项集 Y 出现的条件概率

$$Confidence(X \rightarrow Y) = \mathbb{P}(Y|X) = \frac{\mathbb{P}(X \cap Y)}{\mathbb{P}(X)}$$

关联规则	支持度	置信度
啤酒 → 汽水	1.38%	17.8%
啤酒 → 浆果	0.08%	1.0%
啤酒 → 男士护肤品	0.09%	1.2%

关联模式挖掘

- 提升度（Lift）：在购物篮数据集 $T_1, T_2, \ldots$ 中，当项集 X 出现时，项集 Y 出现的条件概率，相对于 Y 出现的概率的提升

$$Lift(X \to Y) = \frac{Confidence(X \to Y)}{\mathbb{P}(Y)} = \frac{\mathbb{P}(X \cap Y)}{\mathbb{P}(X)\mathbb{P}(Y)}$$

关联规则	支持度	置信度	提升度
啤酒 → 汽水	1.38%	17.8%	1.0
啤酒 → 浆果	0.08%	1.0%	0.3
啤酒 → 男士护肤品	0.09%	1.2%	2.6

关联模式挖掘

-

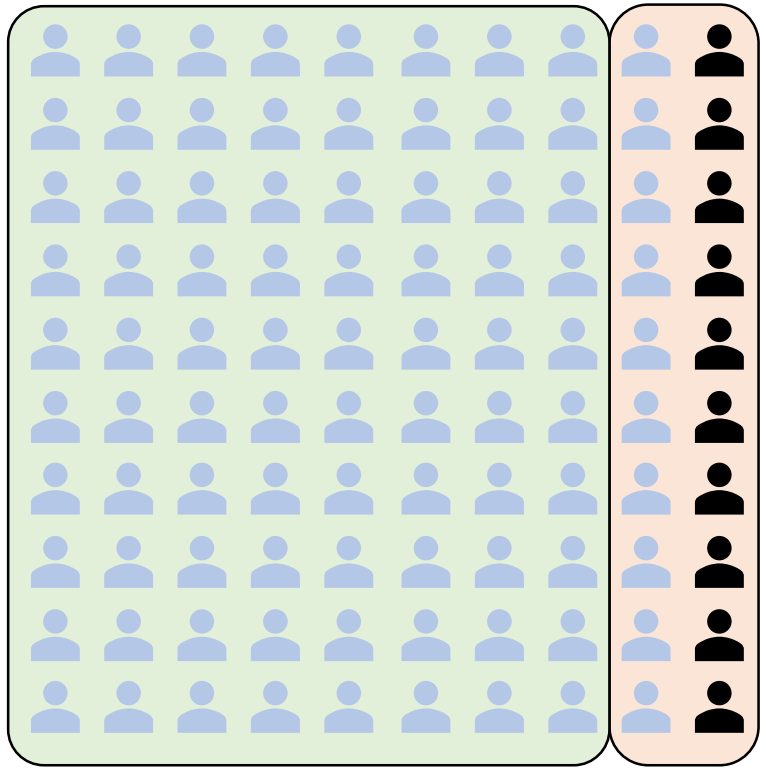
商品	支持度
啤酒	7.77%
汽水	17.44%
浆果	3.32%
男士护肤品	**0.46%**

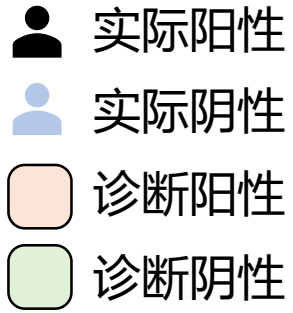
提升了 2.6 倍

关联规则	支持度	置信度	提升度
啤酒 → 汽水	1.38%	17.8%	1.0
啤酒 → 浆果	0.08%	1.0%	0.3
啤酒 → 男士护肤品	0.09%	**1.2%**	**2.6**

关联模式挖掘与二元分类

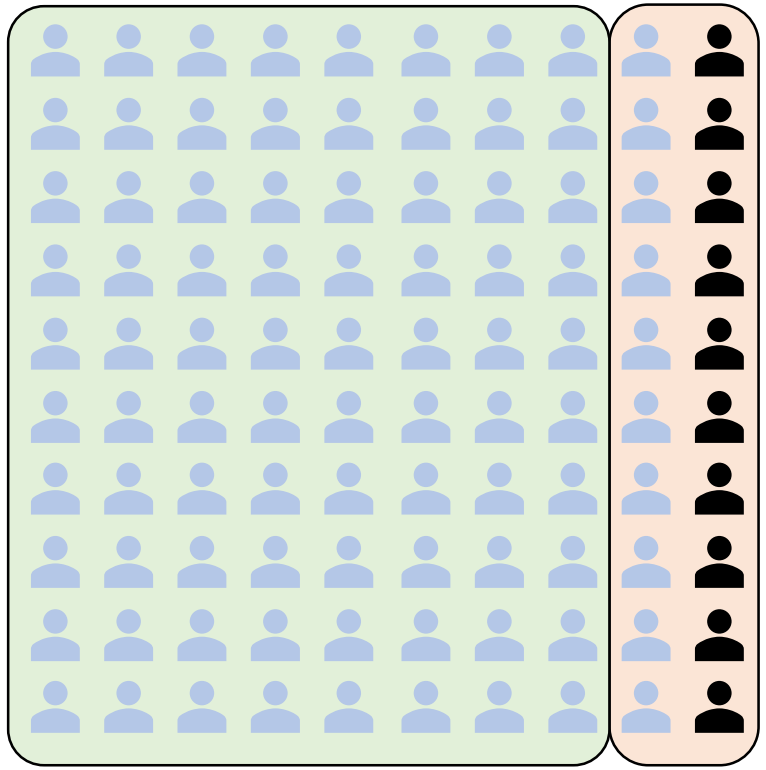
- 回顾：二元分类





关联模式挖掘与二元分类

- 我们把 X 当作“模型”，来预测 Y





关联模式挖掘与二元分类

- 4个数字，或3个百分比

- 支持度，置信度，提升度

关联模式挖掘与故障溯源

- **假如某汽车产品频繁出现某种故障，如何寻找原因?**

- **在数据中挖掘 {?→故障} 或 {故障 → ?}**
  - 假如 {电池电量低→故障} 有较高的提升度……
  - 假如 {空调能耗高→故障} 有较高的提升度……

练习题

假如在超市购物车关联模式挖掘中，我们发现“方便面 $\rightarrow$ 火腿肠”的提升度（Lift）为 1.0。

$$\text{Lift}(X \rightarrow Y) = \frac{\text{Confidence}(X \rightarrow Y)}{\mathbb{P}(Y)} = \frac{\mathbb{P}(Y|X)}{\mathbb{P}(Y)} = \frac{\mathbb{P}(X \cap Y)/\mathbb{P}(X)}{\mathbb{P}(Y)}$$

根据上述数据挖掘结果，以下说法中正确的是_____

A. 买了方便面的顾客通常也同时买了火腿肠

B. 买了火腿肠的顾客通常也同时买了方便面

C. A 和 B 都对

D. A 和 B 都不对

练习题

假如在超市购物车关联模式挖掘中，我们发现“方便面 → 火腿肠”的提升度（Lift）为 1.0。

$$\text{Lift}(X \to Y) = \frac{\text{Confidence}(X \to Y)}{\mathbb{P}(Y)} = \frac{\mathbb{P}(Y|X)}{\mathbb{P}(Y)} = \frac{\mathbb{P}(X \cap Y)/\mathbb{P}(X)}{\mathbb{P}(Y)}$$

根据上述数据挖掘结果，以下说法中正确的是_____

A. 买了方便面的顾客通常也同时买了火腿肠

B. 买了火腿肠的顾客通常也同时买了方便面

C. A 和 B 都对

D. A 和 B 都不对

【答案】：D

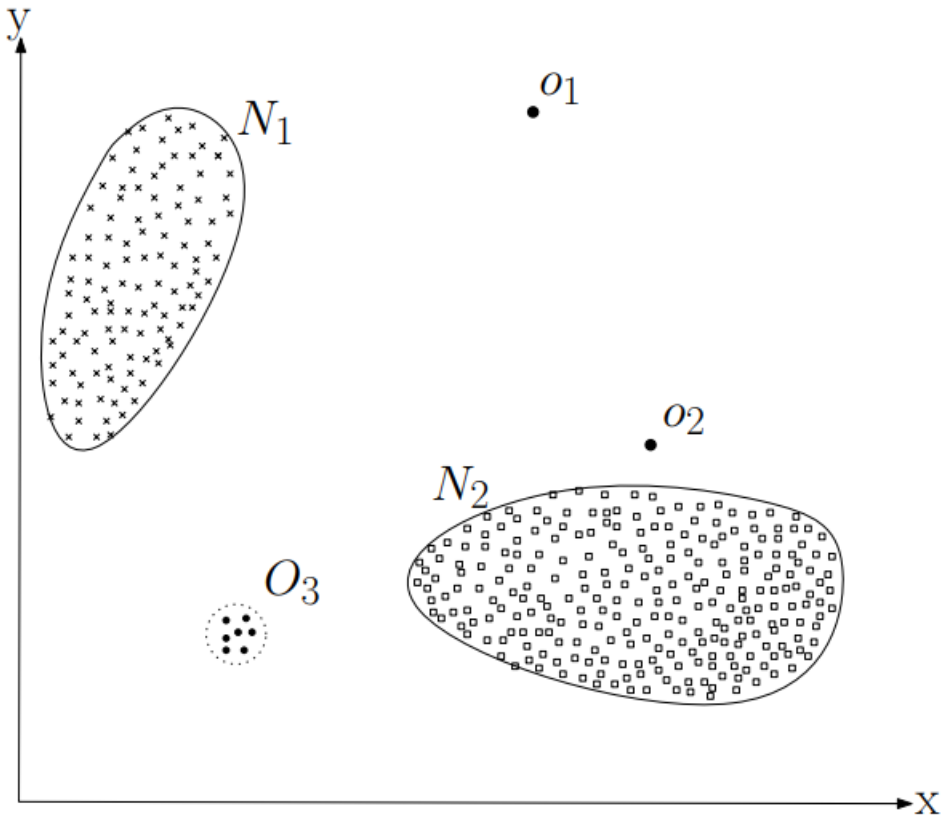
【解析】：“方便面” -> “火腿肠”的提升度为1，即买方便面与买火腿肠两件事相互独立，买不买方便面对买火腿肠的概率没有任何影响。

异常检测

异常检测的应用场景

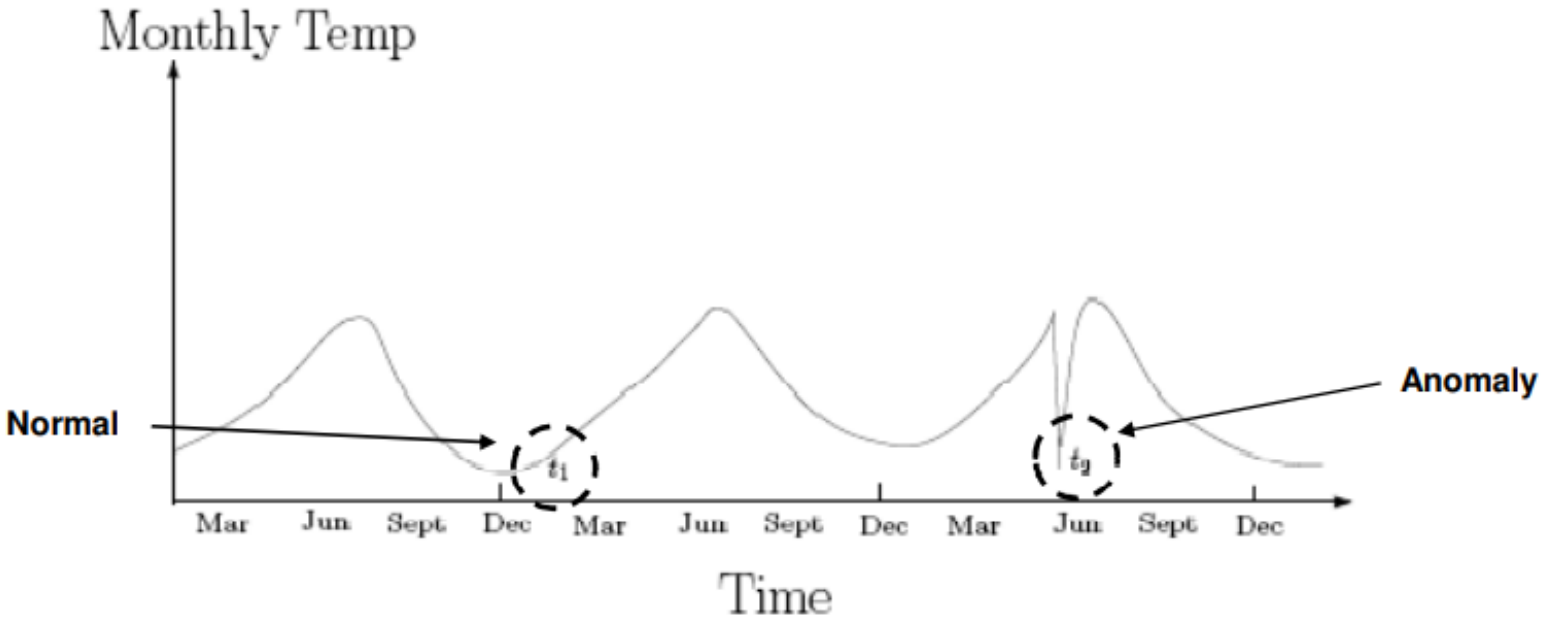
- 设备故障判断
- 银行卡诈骗，保险诈骗
- 网络攻击检测

异常检测的简单示例



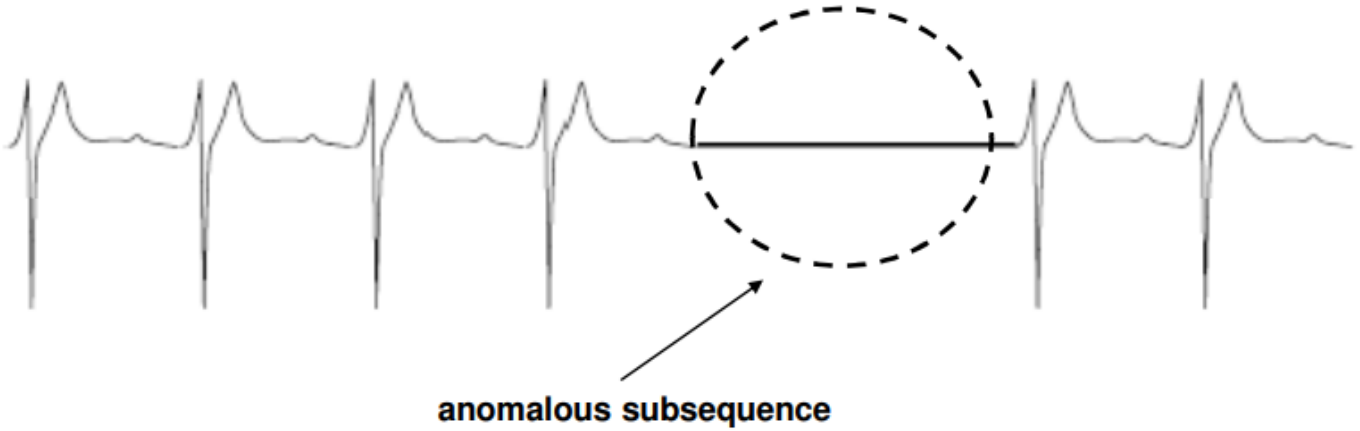
- N1、 N2是正常数据
- o1、 o2和O3是异常

异常检测的简单示例



* Song, et al, "Conditional Anomaly Detection", IEEE Transactions on Data and Knowledge Engineering, 2006.

异常检测的简单示例

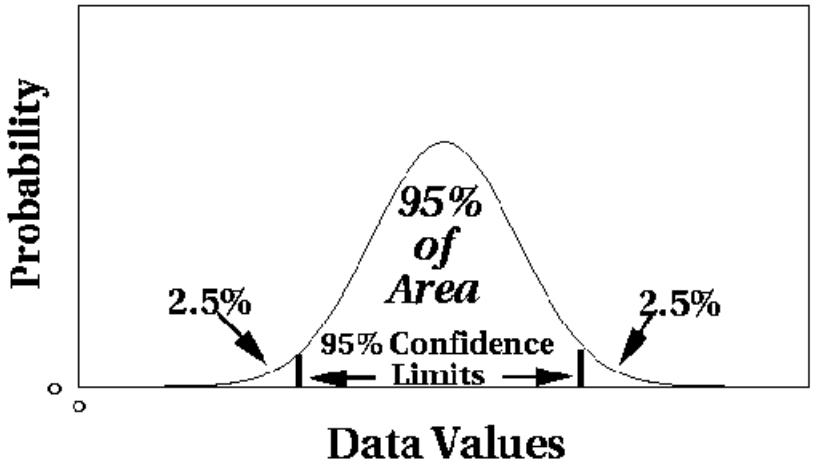


异常检测算法

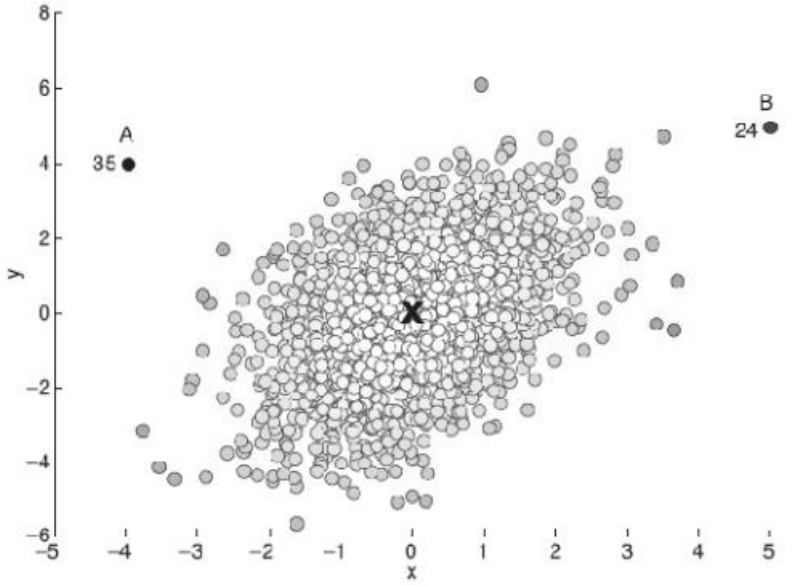
- **针对异常点（高维空间中的点）**
  - 基于统计的方法：找 outliers
  - 基于距离的方法：找离其它点远的点
  - 基于密度的方法：找密度低处的点
  - 基于聚类的方法：找异常的类，或和类的关系异常的点

异常检测算法

- 基于统计的方法：找 outliers



例：基于一维高斯分布的



例：基于多元高斯分布

异常检测算法

- 基于距离的方法：找离其它点远的点

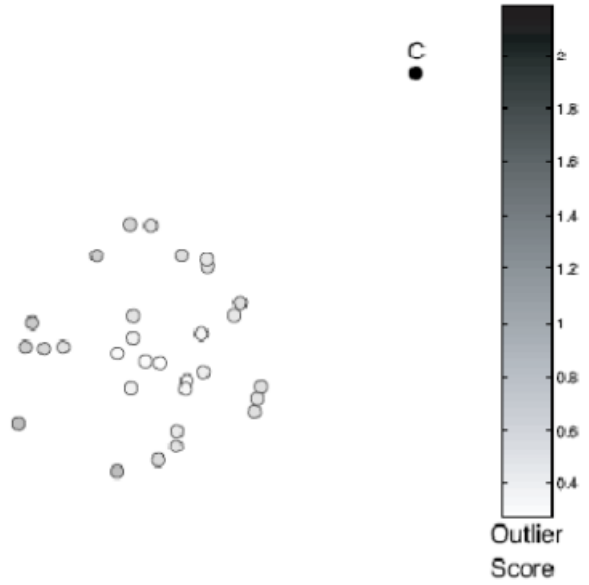


Figure 10.4. Outlier score based on the distance to fifth nearest neighbor.

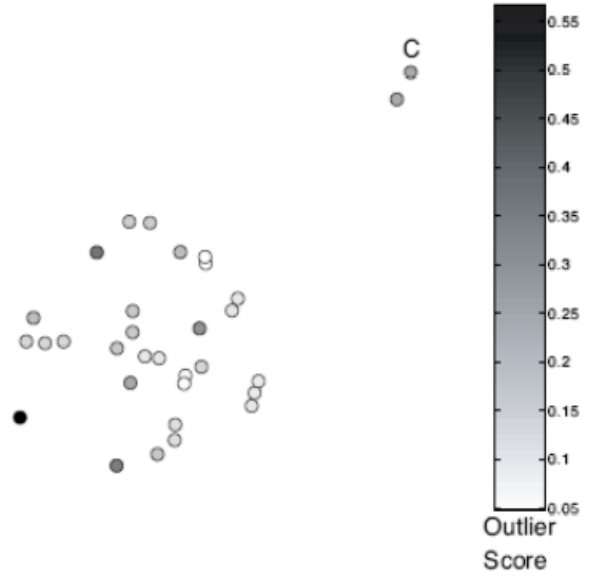
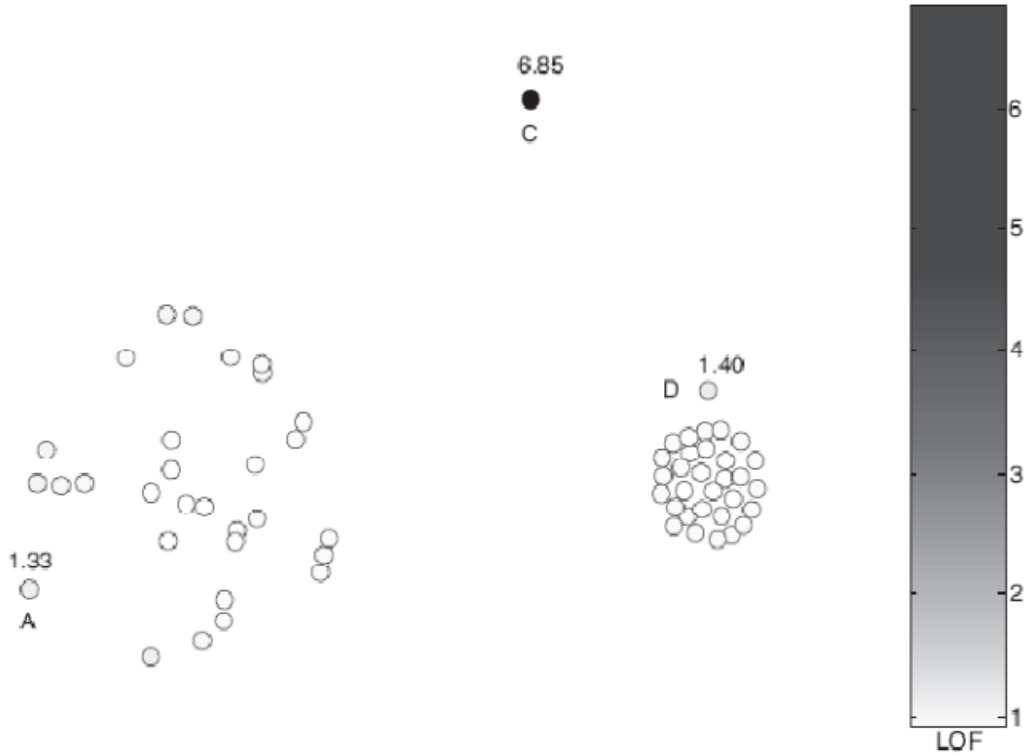


Figure 10.5. Outlier score based on the distance to the first nearest neighbor. Nearby outliers have low outlier scores.

异常检测算法

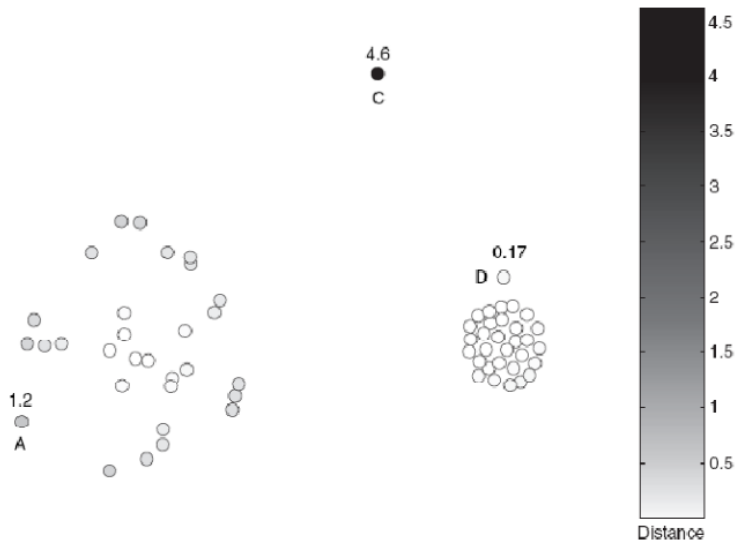
- 基于密度的方法：找密度低处的点



relative density (LOF) outlier scores

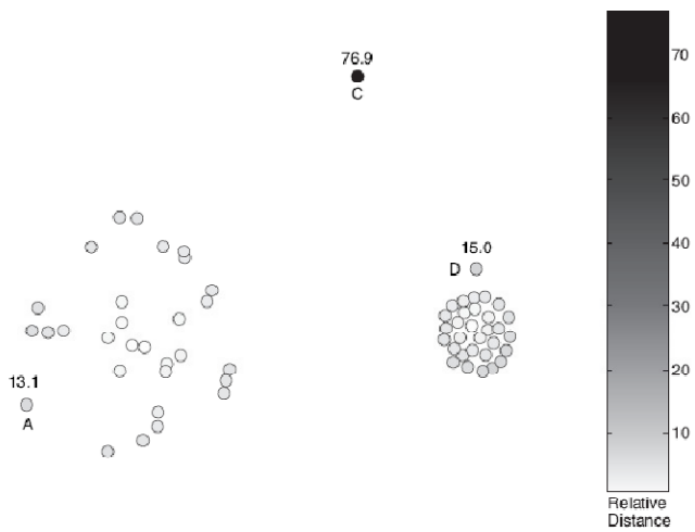
异常检测算法

- 基于聚类的方法：找异常的类，或和类的关系异常的点



distance of points from nearest centroid

$$\frac{\text{distance}(\mathbf{x}, centroid_C)}{\text{median}(\{\forall_{x' \in C}\ \text{distance}(\mathbf{x}', centroid_C)\})}$$



relative distance of points from nearest centroid

异常检测算法

- **异常点（高维空间中的点）检测**
  - 基于统计的方法：找 outliers
  - 基于距离的方法：找离其它点远的点
  - 基于密度的方法：找密度低处的点
  - 基于聚类的方法：找异常的类，或和类的关系异常的点
- **复杂数据（如数据序列）异常检测**
  - 转化为异常点检测问题
  - 针对特殊的数据结构，直接设计检测算法

异常检测算法

- **复杂数据（如数据序列）异常检测：转化为异常点检测问题**
  - 从复杂数据中提取出高维向量
  - 例如：针对序列的特征选择
  - tsfresh - Extract Features on Time Series Easily

